

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Programa de Ingeniería de Sistemas y Computación
HPC

Multiplicación de matrices de forma paralela con OpenMP

Presentado por:

Juan Esteban Cardona Blandón
Jhoan Esteban Corrales Rodríguez
Sebastian Perdomo
Juana Valentina Martínez

Profesor:

Ramiro Andrés Barrios Valencia

Pereira, Colombia

27 de abril de 2026

Índice

1. Marco Teórico	3
1.1. Computación de Alto Rendimiento (HPC)	3
1.2. Fundamentos Matemáticos de la Multiplicación Matricial	3
1.3. Complejidad Computacional de la Multiplicación de Matrices	4
1.4. Programación Secuencial y Concurrente	4
1.5. Sistemas de Memoria Compartida	4
1.6. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria	5
1.7. Optimización Algorítmica y Aprovechamiento de la Caché	5
1.8. Métricas de Rendimiento en Computación Secuencial	6
1.9. Métricas de Rendimiento en Computación Paralela	7
1.10. OpenMP como Modelo de Programación Paralela	8
1.11. Perfilado de Código	9
1.12. Benchmarks de Rendimiento en HPC	10
1.13. Geekbench como Herramienta de Benchmark Multiplataforma	10
2. Marco Conceptual	11
2.1. Características del PC 1	11
2.2. Características del PC 2	12
2.3. Características del software	12
2.3.1. Extracción de parámetros de entrada	13
2.3.2. Núcleo de multiplicación paralela	13
2.3.3. Flujo de ejecución principal	14
3. Perfilado de código	14
3.1. gprof: perfilado por instrumentación del compilador	14
3.2. perf stat: contadores de hardware de la PMU	15
3.3. perf mem: perfilado de accesos a la jerarquía de memoria	17
3.4. perf record: muestreo basado en interrupciones de hardware	18
3.5. Valgrind Massif: perfilado de uso del heap	19
3.6. Valgrind Cachegrind: simulación de la jerarquía de caché	19
3.7. Visión de conjunto y complementariedad	20
3.8. Resultados del perfilado	21
3.8.1. Localización del hotspot	22
3.8.2. Limitaciones de hardware	22
3.8.3. Uso de memoria dinámica	25

3.8.4. Distribución de accesos a memoria (perf mem / IBS)	25
4. Benchmark	26
4.1. Descripción de los Sistemas Evaluados	29
4.2. Resultados de Rendimiento	29
5. Pruebas	30
5.1. Pruebas con OpenMP sin optimización por compilador	31
5.1.1. Pruebas con 2 hilos	31
5.1.2. Pruebas con 4 hilos	31
5.1.3. Pruebas con 6 hilos	32
5.1.4. Pruebas con 8 hilos	32
5.1.5. Pruebas con 12 hilos	33
5.1.6. Gráfica tiempo de ejecución	33
5.1.7. Speedup con OpenMP sin optimización	34
5.2. Pruebas con OpenMP con optimización por compilador	35
5.2.1. Pruebas con 2 hilos	35
5.2.2. Pruebas con 4 hilos	36
5.2.3. Pruebas con 6 hilos	37
5.2.4. Pruebas con 8 hilos	38
5.2.5. Pruebas con 12 hilos	39
5.2.6. Gráfica de tiempo	40
5.2.7. Speedup con OpenMP con optimización	40
5.3. Comparación Pthreads vs OpenMP	41
5.3.1. Resultados sin optimización por compilador	42
5.3.2. Resultados con optimización por compilador	43
6. Conclusiones	45

1. Marco Teórico

1.1. Computación de Alto Rendimiento (HPC)

La Computación de Alto Rendimiento (HPC, por sus siglas en inglés) consiste en la agregación de potencia de cálculo a través de arquitecturas paralelas para resolver problemas científicos y tecnológicos de extrema complejidad. Esta disciplina permite ejecutar algoritmos y modelados matemáticos que resultarían inabarcables para un sistema tradicional debido a restricciones de tiempo y recursos de hardware (Hager & Wellein, 2010). Al dividir cargas masivas de trabajo en instrucciones más pequeñas que se procesan simultáneamente, HPC maximiza la velocidad de procesamiento de datos y la obtención de resultados en la ingeniería (Patterson & Hennessy, 2017).

1.2. Fundamentos Matemáticos de la Multiplicación Matricial

La multiplicación de matrices es una operación algebraica bidimensional fundamental que procesa dos matrices de origen para producir una tercera. Dadas dos matrices A de dimensión $m \times p$ y B de dimensión $p \times n$, su producto da como resultado una matriz C de dimensiones $m \times n$. Matemáticamente, el valor de cada celda individual C_{ij} en la matriz resultante se calcula mediante el producto punto o escalar entre la i -ésima fila de la matriz A y la j -ésima columna de la matriz B . Esta operación se expresa de forma analítica mediante la siguiente sumatoria:

$$C_{ij} = \sum_{k=1}^p A_{ik} B_{kj}$$

Para que esta operación esté matemáticamente definida y sea computacionalmente viable, existe una condición estricta de dimensionalidad: el número de columnas de la primera matriz (p) debe coincidir exactamente con el número de filas de la segunda matriz (Strang, 2016).

En el contexto de la ingeniería y la Computación de Alto Rendimiento (HPC), la multiplicación matricial trasciende la teoría pura para consolidarse como la rutina principal (*kernel*) subyacente en la gran mayoría del software científico. Esta operación constituye el pilar del Nivel 3 de la especificación BLAS (*Basic Linear Algebra Subprograms*), el estándar de facto para operaciones de álgebra lineal. La eficiencia con la que un procesador ejecuta este algoritmo dicta directamente el rendimiento de aplicaciones críticas modernas, tales como el modelado de dinámica de fluidos, la física de partículas, el procesamiento masivo de imágenes y, muy especialmente, el entrenamiento de arquitecturas profundas en inteligencia artificial mediante redes neuronales (Dongarra et al., 1990).

1.3. Complejidad Computacional de la Multiplicación de Matrices

La multiplicación de matrices es un problema canónico en el álgebra lineal computacional, utilizado frecuentemente para evaluar el desempeño y la robustez de las arquitecturas de hardware. Dadas dos matrices cuadradas A y B de dimensiones $n \times n$, el algoritmo iterativo secuencial clásico exige iterar mediante tres ciclos anidados, realizando matemáticamente n^3 multiplicaciones y $n^2(n - 1)$ sumas (Cormen et al., 2009).

Esto establece una complejidad temporal asintótica estricta de $\mathcal{O}(n^3)$. Este crecimiento cúbico implica que, si el tamaño de la matriz se duplica, el esfuerzo computacional se multiplica por un factor de ocho. Debido a esta pronunciada curva de costo algorítmico, la distribución de la carga mediante técnicas de programación paralela (hilos o procesos) se vuelve un requisito indispensable cuando se operan matrices de dimensiones elevadas (Grama et al., 2003).

1.4. Programación Secuencial y Concurrente

La programación secuencial es el paradigma clásico de desarrollo donde las instrucciones de un algoritmo se ejecutan de forma estrictamente lineal, una tras otra, sobre un único núcleo de procesamiento. En este modelo tradicional, el límite de rendimiento está directamente dictado por la frecuencia de reloj del hardware. Como respuesta a estas limitaciones físicas, la programación concurrente estructura el software en múltiples hilos o tareas independientes que hacen progreso de forma superpuesta en el tiempo. Cuando estas tareas operan de manera simultánea en una arquitectura de hardware multinúcleo, la concurrencia se traduce en paralelismo físico, lo cual permite dividir la carga de trabajo y reducir drásticamente el tiempo de ejecución en problemas de cómputo intensivo (Ben-Ari, 2006).

1.5. Sistemas de Memoria Compartida

En el ámbito de la computación paralela, el modelo de memoria compartida establece que múltiples hilos de ejecución (como los creados por la librería Pthreads) operan dentro de un mismo espacio de direccionamiento lógico. A nivel de arquitectura, esto significa que todos los núcleos del procesador pueden acceder de forma directa y simultánea a las mismas estructuras de datos residentes en la memoria principal. Para problemas matriciales, este modelo resulta excepcionalmente eficiente frente a los sistemas de memoria distribuida, ya que evita la necesidad de copiar y transmitir los datos de las matrices de origen repetidamente entre procesos. Aunque este paradigma exige control riguroso para evitar condiciones de carrera, en operaciones intrínsecamente independientes como el cálculo individual de las celdas de una

matriz resultado, su implementación maximiza la eficiencia computacional sin sobrecarga de sincronización (Silberschatz et al., 2018).

1.6. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria

El rendimiento de una aplicación de alto rendimiento (HPC) no depende exclusivamente de la capacidad matemática del procesador, sino también del acceso al subsistema de memoria. Para mitigar la alta latencia térmica y física de la memoria RAM, los procesadores modernos integran una jerarquía de memoria ultrarrápida cercana a los núcleos, conocida como memoria Caché, habitualmente dividida en niveles (L1, L2 y L3). La unidad mínima de transferencia de información entre la RAM principal y la memoria caché no es un byte aislado, sino un bloque continuo de datos denominado línea de caché (*cache line*), que en las arquitecturas modernas suele constar de 64 bytes (Drepper, 2007).

Esta transferencia por bloques está fundamentada en el principio de localidad espacial de memoria, un concepto empírico que dicta que si un programa accede a una dirección de memoria, es inminentemente probable que acceda a las direcciones adyacentes a continuación. En lenguajes de programación como C, las matrices bidimensionales se almacenan en la memoria RAM de manera contigua organizadas por filas (*row-major order*). Durante la multiplicación matricial clásica iterativa, la segunda matriz se recorre inevitablemente por columnas; este patrón de acceso ortogonal rompe por completo la localidad espacial. Al intentar leer datos saltando grandes bloques de memoria, los valores necesarios no se encuentran en la línea de caché cargada, provocando fallos de caché (*cache misses*) que obligan al procesador a detenerse (*stall*) mientras busca los datos nuevamente en la lenta memoria RAM, lo cual penaliza severamente el desempeño del algoritmo (Patterson & Hennessy, 2017).

1.7. Optimización Algorítmica y Aprovechamiento de la Caché

Aunque la paralelización distribuye la carga de trabajo, el cuello de botella de la memoria solo puede resolverse reestructurando el algoritmo subyacente. La estrategia de optimización más directa para la multiplicación de matrices en C consiste en el intercambio de bucles (*loop interchange*). Altera el orden tradicional de los ciclos anidados de i, j, k a i, k, j . Matemáticamente, el resultado es idéntico, pero a nivel de hardware, el bucle más interno ahora recorre las matrices B y C estrictamente por filas. Esto garantiza que el procesador consuma por completo los 64 bytes de cada línea de caché (*cache line*) cargada antes de solicitar la siguiente, aprovechando al máximo la localidad espacial y reduciendo las penalizaciones por fallos de caché en órdenes de magnitud (Lam et al., 1991).

Para matrices masivas que exceden la capacidad total de la memoria caché (L1 y L2),

la literatura de HPC emplea una técnica avanzada denominada multiplicación por bloques o *Loop Tiling*. Este método particiona las matrices originales en submatrices (o bloques) de un tamaño cuidadosamente calculado para que residan por completo dentro de la caché rápida del procesador. En lugar de multiplicar filas y columnas completas, el algoritmo opera sobre estos bloques aislados, reutilizando los datos cargados repetidas veces antes de que el controlador de memoria los descarte (*eviction*). Esta técnica transforma un problema limitado por el ancho de banda de la memoria (*memory-bound*) en uno puramente computacional (*compute-bound*) (Grama et al., 2003).

1.8. Métricas de Rendimiento en Computación Secuencial

El análisis de un algoritmo secuencial requiere establecer una línea base absoluta sobre la cual evaluar cualquier mejora algorítmica o de hardware. La métrica fundamental empírica es el Tiempo de Ejecución (*Wall-clock time*), que representa el tiempo físico real transcurrido desde el inicio hasta la finalización del programa. Sin embargo, dado que este valor fluctúa dependiendo de la interferencia del sistema operativo y los accesos a memoria, la arquitectura de computadores recurre a métricas de rendimiento intrínseco (*throughput*) (Patterson & Hennessy, 2017).

Para cargas de trabajo de alto rendimiento algebraico como la multiplicación de matrices, la métrica estandarizada predominante es el volumen de Operaciones de Punto Flotante por Segundo (FLOPS, por sus siglas en inglés). Esta medida cuantifica el rendimiento matemático bruto del procesador y se define mediante la siguiente relación:

$$\text{FLOPS} = \frac{\text{Operaciones Aritméticas Totales}}{T_{\text{ejecución}}} \quad (1)$$

Para el algoritmo clásico iterativo que multiplica dos matrices cuadradas de dimensión $n \times n$, el número total de operaciones de punto flotante se establece analíticamente en $2n^3$ (contabilizando una instrucción de multiplicación y una de suma por cada iteración del bucle más interno). Al contrastar los FLOPS experimentales obtenidos en la ejecución secuencial contra el rendimiento teórico máximo (*Peak FLOPS*) que el fabricante del procesador garantiza para un solo núcleo, es posible diagnosticar ineficiencias de memoria o cuellos de botella antes de recurrir a la paralelización (Hager & Wellein, 2010).

Adicionalmente, el rendimiento de la ejecución en un único núcleo se evalúa a nivel microarquitectónico mediante el IPC (Instrucciones por Ciclo de reloj). Esta métrica indica la eficiencia con la que el código compilado logra mantener ocupada la segmentación (*pipeline*) del procesador, minimizando las detenciones o burbujas producidas por dependencias de datos en la ejecución lineal (Yi et al., 2020).

1.9. Métricas de Rendimiento en Computación Paralela

Para cuantificar el éxito de una implementación paralela y su eficiencia frente a la ejecución secuencial, la literatura científica emplea métricas matemáticas estandarizadas. La métrica fundamental es el *Speedup* o Aceleración (S_p), la cual mide la ganancia de velocidad relativa. Se define formalmente como:

$$S_p = \frac{T_1}{T_p} \quad (2)$$

donde T_1 representa el tiempo de ejecución del algoritmo secuencial óptimo sobre un único núcleo, y T_p es el tiempo de ejecución del algoritmo paralelo empleando p unidades de procesamiento (hilos o procesos). Derivada de esta métrica se encuentra la Eficiencia (E_p), que evalúa el grado de aprovechamiento de los recursos físicos, indicando qué fracción del tiempo los procesadores realizan trabajo útil sin quedar ociosos:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad (3)$$

El desempeño asintótico de estos sistemas está regido por dos leyes analíticas que abordan la escalabilidad desde diferentes perspectivas teóricas. La **Ley de Amdahl** evalúa la *escalabilidad fuerte*, demostrando que el *Speedup* máximo de un sistema está estrictamente limitado por la fracción de código que no se puede paralelizar. Asumiendo un tamaño de problema fijo, la ley se expresa como:

$$S_{\text{Amdahl}} \leq \frac{1}{(1 - f) + \frac{f}{p}} \quad (4)$$

donde f es la fracción del programa que es paralelizable y $(1 - f)$ es la porción estrictamente secuencial (como la inicialización de memoria mediante `malloc` o la lectura/escritura en disco). Según este postulado matemático, incluso si el número de procesadores p tiende a infinito, la aceleración máxima nunca superará $1/(1 - f)$. Esto explica el rendimiento decreciente observado cuando el uso de múltiples hilos lógicos en un procesador alcanza su límite físico y no aporta mejoras significativas (Patterson & Hennessy, 2017).

En contraste, la **Ley de Gustafson-Barsis** describe la *escalabilidad débil*. Esta ley argumenta que, en la práctica profesional, los ingenieros no buscan resolver un problema fijo más rápido, sino que utilizan el mayor poder de cómputo para resolver problemas de un tamaño sustancialmente mayor en el mismo marco de tiempo (Gustafson, 1988). Su modelo se define como:

$$S_{\text{Gustafson}} = (1 - f) + p \cdot f \quad (5)$$

Bajo la visión de Gustafson, a medida que la dimensión de las matrices crece de forma masiva, la cantidad de operaciones paralelas f (multiplicaciones flotantes) aumenta drásticamente, haciendo que la fracción secuencial inicial $(1 - f)$ pierda peso relativo. Esto mitiga el cuello de botella pronosticado por Amdahl, permitiendo alcanzar un *Speedup* escalable y un uso altamente eficiente de los recursos al ajustar la carga de trabajo proporcionalmente al hardware.

1.10. OpenMP como Modelo de Programación Paralela

OpenMP (*Open Multi-Processing*) es una interfaz de programación de aplicaciones (API) de estándar abierto para la programación paralela en sistemas de memoria compartida, desarrollada y mantenida por el consorcio OpenMP Architecture Review Board. El estándar define un conjunto de directivas de compilador, rutinas de biblioteca en tiempo de ejecución y variables de entorno que permiten al programador expresar paralelismo a nivel de bucles y tareas directamente sobre un código fuente C, C++ o Fortran existente, sin necesidad de reescribir la arquitectura del programa (Dagum & Menon, 1998). Esta característica lo convierte en el modelo predominante para la paralelización incremental en HPC sobre arquitecturas multinúcleo de memoria compartida.

El modelo de ejecución de OpenMP se fundamenta en el paradigma de bifurcación y unión (*fork-join*). El programa inicia su ejecución como un hilo único denominado hilo maestro, el cual ejecuta el código secuencial de forma ordinaria. Al encontrar una directiva de región paralela, el hilo maestro bifurca un equipo de hilos de trabajo que ejecutan el bloque de código encerrado de forma concurrente; al concluir dicha región, todos los hilos se sincronizan en una barrera implícita y se unen de nuevo al hilo maestro, que retoma la ejecución secuencial (Dagum & Menon, 1998). Este ciclo puede repetirse tantas veces como regiones paralelas declare el programa, lo que permite una adopción gradual y controlada del paralelismo sobre una base de código secuencial ya validada.

La directiva central para la paralelización de bucles es `#pragma omp parallel for`, que distribuye automáticamente las iteraciones del bucle entre los hilos del equipo. El programador puede controlar la política de distribución de trabajo mediante la cláusula `schedule`, que admite los modos estático (*static*), dinámico (*dynamic*) y guiado (*guided*), cada uno con implicaciones distintas sobre el balance de carga y la sobrecarga de coordinación entre hilos (Chapman et al., 2008). Para la multiplicación de matrices, el modo estático resulta óptimo dado que las iteraciones presentan una carga de trabajo homogénea; esto permite al compilador asignar bloques contiguos de filas a cada hilo sin incurrir en la sobrecarga de planificación dinámica en tiempo de ejecución.

El modelo de datos de OpenMP clasifica las variables del programa en dos categorías:

compartidas (*shared*), accesibles y modificables por todos los hilos del equipo simultáneamente, y privadas (*private*), de las cuales cada hilo posee una copia independiente. El control explícito sobre este ámbito de visibilidad es un requisito de corrección del programa paralelo: las matrices de entrada y la matriz resultado deben declararse como compartidas para que todos los hilos lean y escriban sobre la misma estructura de datos, mientras que las variables de índice de los bucles internos deben ser privadas para evitar condiciones de carrera (Chapman et al., 2008). Dado que cada celda C_{ij} de la matriz resultado es calculada de forma completamente independiente por un único hilo, la multiplicación de matrices no requiere mecanismos de sincronización explícita como secciones críticas o operaciones atómicas, lo que elimina la principal fuente de sobrecarga en implementaciones OpenMP y permite un escalado cercano al teórico.

1.11. Perfilado de Código

El perfilado de código es la disciplina de la ingeniería de software que mide empíricamente el comportamiento dinámico de un programa en ejecución, con el objetivo de identificar con precisión cuáles funciones o regiones del código concentran el mayor consumo de tiempo de CPU o de recursos de memoria. En el contexto de HPC, esta técnica constituye el primer paso metodológico indispensable antes de cualquier intento de optimización o paralelización, ya que evita el anti-patrón de optimizar secciones del código que no representan cuellos de botella reales (Hager & Wellein, 2010).

Los perfiladores operan bajo dos paradigmas fundamentales. La primera estrategia es la *instrumentación*, que consiste en insertar sondas de medición en puntos específicos del código fuente o del binario compilado; este enfoque proporciona datos exactos pero puede introducir una sobrecarga de ejecución significativa. La segunda estrategia es el *muestreo estadístico*, en la cual la herramienta interrumpe la ejecución del proceso a intervalos regulares para registrar el estado del contador de programa; este método produce resultados estadísticamente representativos con una perturbación mínima sobre el tiempo de ejecución real (Grama et al., 2003).

Entre las herramientas más ampliamente adoptadas en entornos HPC sobre Linux se destacan dos: gprof (GNU Profiler) y perf. gprof opera combinando instrumentación en tiempo de compilación con muestreo temporal; analiza el binario para generar un reporte jerarquizado que muestra el tiempo acumulado por función y el grafo de llamadas, permitiendo identificar con granularidad de función dónde el programa concentra la mayor fracción de su tiempo de cómputo (Graham et al., 1982a). perf, por su parte, es una herramienta de análisis integrada al núcleo Linux que accede directamente a los contadores de hardware del procesador mediante la interfaz de eventos de rendimiento del sistema operativo. Esto le permite recolectar

métricas microarquitectónicas de bajo nivel, como la tasa de fallos de caché L1/L2/L3, el número de predicciones erróneas de salto, el IPC real y el número de instrucciones retiradas por ciclo; datos que resultan imposibles de obtener mediante perfilado por instrumentación de código fuente (Patterson & Hennessy, 2017).

1.12. Benchmarks de Rendimiento en HPC

Un benchmark de rendimiento es un procedimiento de medición estandarizado y reproducible que ejecuta una carga de trabajo representativa sobre un sistema con el propósito de obtener métricas comparables entre diferentes arquitecturas de hardware, configuraciones de software o implementaciones algorítmicas. Para que un benchmark posea validez científica, debe cumplir criterios de representatividad (que el problema resuelto sea relevante para las aplicaciones de interés), reproducibilidad (resultados consistentes bajo las mismas condiciones) y equidad (que no favorezca artificialmente ninguna arquitectura específica) (Dongarra et al., 1990).

En el dominio de HPC, el benchmark de referencia histórico y más influyente es LINPACK (*Linear Algebra Package*), desarrollado por Jack Dongarra, y su sucesor de alta precisión HPL (*High Performance LINPACK*). El HPL resuelve un sistema denso de ecuaciones lineales de la forma $Ax = b$ de dimensión $N \times N$ mediante la factorización LU con pivoteo parcial, reportando el rendimiento efectivo en GFLOPS ($R_{\text{máx}}$) y contrastándolo contra el rendimiento teórico máximo del hardware (R_{peak}). La eficiencia HPL se define como el cociente $R_{\text{máx}}/R_{\text{peak}}$, y constituye el criterio de clasificación oficial de la lista TOP500, el ranking mundial de supercomputadoras (Dongarra et al., 1990). Para el benchmarking de operaciones de álgebra lineal densa a nivel de núcleo computacional, el estándar utilizado en la literatura científica es DGEMM (*Double-precision General Matrix Multiply*), que forma parte de la especificación BLAS Nivel 3 y mide directamente el rendimiento de la multiplicación de matrices en doble precisión, sirviendo como indicador canónico del rendimiento sostenido de punto flotante de un procesador (Patterson & Hennessy, 2017).

1.13. Geekbench como Herramienta de Benchmark Multiplataforma

Geekbench es una suite de benchmarks multiplataforma desarrollada por Primate Labs, diseñada para evaluar el rendimiento del procesador y la memoria de un sistema mediante cargas de trabajo sintéticas representativas de aplicaciones reales del mundo moderno. A diferencia de los benchmarks orientados exclusivamente al cómputo numérico como HPL o DGEMM, Geekbench adopta un enfoque holístico que cubre categorías como compresión

de datos, procesamiento de imágenes, renderizado de documentos, compilación de código y cargas de trabajo de aprendizaje automático, con el objetivo de capturar el comportamiento del procesador frente a una gama amplia de patrones de acceso a memoria y tipos de instrucciones (Primate Labs Inc., 2024).

La versión actual, Geekbench 6, organiza la evaluación del procesador en dos secciones principales: núcleo único y múltiples núcleos. Cada sección se subdivide en cargas de trabajo de aritmética entera y aritmética de punto flotante. La puntuación compuesta final se calcula como la media aritmética ponderada de las puntuaciones de subsección, asignando un peso del 65 % a las cargas enteras y el 35 % restante a las de punto flotante. Las puntuaciones están calibradas contra una línea base de 2,500 puntos correspondiente a un procesador Intel Core i7-12700, de modo que una puntuación doble representa exactamente el doble del rendimiento de referencia (Primate Labs Inc., 2024). Geekbench 6 introduce además un modelo de paralelismo basado en tareas compartidas en sustitución del esquema de tareas independientes de versiones anteriores, lo cual modela con mayor fidelidad el comportamiento real de las aplicaciones modernas que distribuyen trabajo entre núcleos (Primate Labs Inc., 2024).

Desde una perspectiva microarquitectónica, el documento técnico oficial de Geekbench 6 caracteriza cada carga de trabajo mediante métricas de bajo nivel que incluyen IPC, tasa de fallos de predicción de salto, tamaño del conjunto de trabajo activo y tasas de fallos de caché para los niveles L1D, L2 y L3. Por ejemplo, cargas de trabajo con patrones de acceso altamente irregulares como la carga de navegación basada en el algoritmo de Dijkstra registran tasas de fallo en L3 del 23.9 % en modo de núcleo único, mientras que cargas con alta localidad espacial como la compresión de datos exhiben tasas de fallo L3 del 12.8 %, evidenciando cómo el comportamiento de caché impacta directamente la puntuación final del benchmark (Primate Labs Inc., 2024).

2. Marco Conceptual

2.1. Características del PC 1

- Procesador: Ryzen 5 7600X
- Cantidad de núcleos: 6
- Cantidad de subprocesos/hilos: 12
- Frecuencia básica del procesador: 4.7GHz

- Frecuencia Turbo máxima: 5.3GHz
- Memoria RAM: 32GB DDR5 @ 5200 MHz
- L1 Instruction Cache 32.0 KB x 6
- L1 Data Cache 32.0 KB x 6
- L2 Cache 1.00 MB x 6
- L3 Cache 32.0 MB
- Sistema operativo: Arch Linux

2.2. Características del PC 2

- Procesador: Intel Core i7-11800H
- Cantidad de núcleos: 8
- Cantidad de subprocesos/hilos: 16
- Frecuencia básica del procesador: 4.6GHz
- Frecuencia Turbo máxima: 5.0GHz
- Memoria RAM: 16GB DDR4 @ 3200 MHz
- L1 Instruction Cache 32.0 KB x 8
- L1 Data Cache 48.0 KB x 8
- L2 Cache 1.25 MB x 8
- L3 Cache 24.0 MB
- Sistema operativo: Debian GNU/Linux 12

2.3. Características del software

Se puede encontrar el código fuente del proyecto en el siguiente repositorio de GitHub:
<https://github.com/Juanes7222/HPC-Matrix.git>

El programa implementa la multiplicación de matrices cuadradas de orden n haciendo uso de paralelismo a nivel de hilos mediante la biblioteca OpenMP. Su estructura se organiza en tres componentes principales: la extracción y validación de argumentos de entrada, el núcleo de cómputo paralelo, y la orquestación general del flujo de ejecución en la función principal.

2.3.1. Extracción de parámetros de entrada

La función `extractThreadCount` se encarga de leer el segundo argumento de la línea de comandos, que corresponde al número de hilos deseado. Si dicho argumento está ausente o su valor no es un entero positivo, el programa termina de forma controlada emitiendo un mensaje de error descriptivo.

2.3.2. Núcleo de multiplicación paralela

El corazón del programa reside en la función de multiplicación paralela, que recibe dos matrices de enteros de tamaño $n \times n$ junto con la matriz de salida. La multiplicación sigue el algoritmo clásico de producto matricial, donde cada elemento `result[i][j]` se calcula como la suma acumulada

$$\sum_{k=0}^{n-1} \text{matrix1}[i][k] \cdot \text{matrix2}[k][j].$$

La directiva de paralelización distribuye los bucles externos sobre los índices i y j entre los hilos disponibles. La cláusula `collapse(2)` fusiona ambos bucles en un único espacio de iteración de tamaño n^2 , lo que incrementa la granularidad del trabajo disponible para el planificador de OpenMP y mejora el balance de carga, especialmente cuando n es pequeño en relación con el número de hilos. La distribución estática de iteraciones mediante `schedule(static)` es óptima para cargas de trabajo homogéneas como la multiplicación densa de matrices, donde cada iteración implica exactamente n operaciones de multiplicación-acumulación.

El manejo del alcance de variables se hace explícito mediante `default(none)`, que obliga al programador a declarar el rol de cada variable. Las matrices se declaran como compartidas, dado que todos los hilos acceden a las mismas estructuras en memoria sin conflictos de escritura sobre las matrices de entrada, y con escrituras disjuntas sobre la matriz de salida, ya que cada hilo escribe en un elemento $[i][j]$ único. La variable escalar n se declara como `firstprivate`, proveyendo a cada hilo una copia local inicializada con el valor original y evitando accesos concurrentes al mismo objeto en memoria.

Adicionalmente, la directiva de desenrollado parcial instruye al entorno de ejecución de OpenMP 6.1 para desenrollar el bucle interno en grupos de cuatro iteraciones. Este desenrollado reduce la sobrecarga de las instrucciones de control de bucle y expone mayor paralelismo a nivel de instrucción al pipeline del procesador, permitiendo que la unidad de ejecución emita múltiples operaciones de multiplicación-acumulación de forma simultánea.

2.3.3. Flujo de ejecución principal

La función principal orquesta el flujo completo del programa. Primero extrae el tamaño de la matriz n y el número de hilos desde los argumentos de entrada. Como medida de seguridad, si el número de hilos solicitado supera n , se recorta a ese valor, evitando la asignación de hilos sin trabajo útil que generarían sobrecarga innecesaria.

A continuación, las tres matrices se asignan dinámicamente y las dos matrices de entrada se inicializan con valores aleatorios. El tiempo de ejecución se mide con precisión usando un reloj monotónico, que provee una referencia de tiempo no sujeta a ajustes del sistema y garantiza mediciones de latencia fiables. Tras la multiplicación paralela, el tiempo transcurrido se reporta en milisegundos y las tres matrices se liberan de la memoria dinámica, evitando fugas de memoria.

3. Perfilado de código

El perfilado de código (*profiling*) es el proceso de medir el comportamiento de un programa en ejecución con el objetivo de identificar dónde se concentra el costo computacional y de memoria. A diferencia del análisis estático, el perfilado opera sobre la ejecución real del programa, capturando métricas como tiempo por función, contadores de hardware del procesador, comportamiento de la jerarquía de caché y consumo dinámico de memoria. Esto permite construir evidencia empírica que justifica las decisiones de optimización y sirve de línea base para comparar implementaciones alternativas.

El script `bench_profiling.sh` orquesta seis técnicas de perfilado complementarias sobre la implementación secuencial de multiplicación de matrices, para los tamaños $N \in \{128, 256, 512, 1024\}$. Cada técnica responde una pregunta distinta: cuánto tiempo tarda, en qué función se acumula ese tiempo, qué eventos de hardware ocurren, cómo se comporta la jerarquía de caché y cuánta memoria dinámica se consume. Los resultados de todas las herramientas se consolidan en un archivo CSV (`data_profiling.csv`) que sirve de entrada al generador de reportes.

3.1. gprof: perfilado por instrumentación del compilador

`gprof` es una herramienta de perfilado basada en instrumentación generada en tiempo de compilación (Graham et al., 1982b). El binario `mul_seq_gprof` se compila con las flags `-pg` y `-fno-inline`:

- `-pg`: instruye al compilador para insertar una llamada a la función `mcount()` al inicio de cada función del programa. Durante la ejecución, estas llamadas actualizan en tiempo

real los contadores de frecuencia de invocación y los acumuladores de tiempo en un archivo binario llamado `gmon.out`.

- **-fno-inline**: impide que el compilador reemplace el cuerpo de las funciones directamente en sus sitios de llamada (*inlining*).

El reporte que produce `gprof` tiene dos partes principales:

- **Flat profile**: lista cada función con el porcentaje del tiempo total que le corresponde, los segundos acumulados, el número de veces que fue invocada y el tiempo promedio por llamada. Este perfil responde directamente la pregunta “¿en qué función pasa el programa la mayor parte del tiempo?”.
- **Call graph**: muestra la jerarquía de llamadas entre funciones y cuánto tiempo se propaga desde cada función hija hacia su función padre. Es útil para distinguir si una función es costosa por su propia lógica o porque invoca a subfunciones costosas.

La limitación de `gprof` es que su medición del tiempo se basa en muestreo estadístico del contador de programa (*program counter*) con una resolución de aproximadamente 10 ms. Para programas de corta duración o con alta granularidad de funciones los porcentajes pueden ser estadísticamente ruidosos. Además, al requerir recompilación con instrumentación, el binario perfilado no es idéntico al binario de producción, lo que puede alterar el comportamiento de caché y el layout de código.

3.2. perf stat: contadores de hardware de la PMU

`perf stat` accede directamente a la *Performance Monitoring Unit* (PMU), un conjunto de registros físicos presentes en el procesador diseñados específicamente para contar eventos de hardware con un overhead mínimo sobre el programa medido.

El conjunto de eventos medidos cubre cuatro categorías de cuello de botella típicos en el contexto de la multiplicación de matrices:

Throughput del pipeline. Los eventos `cycles` e `instructions` permiten calcular el *Instructions Per Cycle* (IPC):

$$\text{IPC} = \frac{\text{instructions}}{\text{cycles}}. \quad (6)$$

Un IPC cercano a 1 indica que por cada ciclo de reloj el procesador completa en promedio una instrucción. Valores menores a 1 señalan que el pipeline sufre *stalls* frecuentes, generalmente

por esperas de memoria. Valores mayores a 2 indican que el procesador está emitiendo múltiples instrucciones por ciclo de forma efectiva gracias al paralelismo a nivel de instrucción (ILP).

Caché de último nivel (LLC). Los eventos `cache-references` y `cache-misses` miden la presión sobre la LLC (*Last Level Cache*). La tasa de fallo

$$\text{LLC miss rate} = \frac{\text{cache-misses}}{\text{cache-references}} \times 100 \% \quad (7)$$

indica qué fracción de los accesos a la LLC no encontró el dato solicitado y tuvo que recurrir a la memoria principal (DRAM). En multiplicación de matrices, para tamaños grandes de N , las tres matrices no caben simultáneamente en LLC y esta tasa escala drásticamente, siendo el principal responsable de la degradación del rendimiento con respecto al pico teórico del procesador.

Caché L1 de datos (L1-D). Los eventos `L1-dcache-loads` y `L1-dcache-load-misses` miden la localidad efectiva del acceso a datos en el nivel de caché más cercano al procesador. La tasa de fallo en L1-D es el indicador más sensible al orden de los bucles en el algoritmo: el orden i - j - k produce accesos no contiguos a la columna `matrix2[k][j]`, penalizando directamente este contador.

Translation Lookaside Buffer (TLB). Los eventos `dTLB-loads` y `dTLB-load-misses` miden el costo de la traducción de direcciones virtuales a físicas. El TLB es una caché especializada de entradas de la tabla de páginas del sistema operativo. Cuando se trabaja con matrices grandes cuyos elementos están distribuidos en muchas páginas de memoria virtual, los fallos de TLB agregan latencia adicional independiente de la jerarquía de caché de datos. Esta métrica permite separar si la degradación de rendimiento se debe a presión de caché o a presión de TLB.

Predictor de saltos. Los eventos `branch-instructions` y `branch-misses` cuantifican la efectividad del predictor de saltos del procesador. En un algoritmo de multiplicación de matrices con bucles perfectamente regulares se espera una tasa de fallo del predictor cercana a cero, ya que todos los saltos son altamente predecibles. Un valor elevado sería indicativo de lógica condicional inesperada en el código o de fallos en la vectorización automática.

3.3. perf mem: perfilado de accesos a la jerarquía de memoria

`perf mem` es una extensión especializada de `perf` diseñada específicamente para analizar el comportamiento de los accesos a memoria, distinguiendo en qué nivel de la jerarquía fue satisfecho cada acceso (*memory hierarchy attribution*). Mientras que `perf stat` reporta tasas de fallo de caché como totales globales y `perf record` localiza funciones costosas, `perf mem` responde una pregunta más precisa: “¿desde qué nivel de la jerarquía de memoria se están sirviendo los accesos de carga del programa?”

El script ejecuta la herramienta en dos fases:

- **perf mem record**: captura muestras de eventos de acceso a memoria mediante los registros PEBS (*Precise Event Based Sampling*) disponibles en procesadores Intel, o su equivalente en AMD. A diferencia del muestreo por ciclos de `perf record`, PEBS captura el evento en el ciclo exacto en que ocurre la instrucción de carga, lo que elimina el *skid* (desplazamiento entre el evento real y la muestra registrada) y permite atribuir con precisión cada acceso a la instrucción que lo causó.
- **perf mem report -sort=mem**: formatea el reporte agrupando las muestras por tipo de acceso a memoria. La opción `-t load` restringe el análisis a operaciones de lectura, que son las dominantes en la multiplicación de matrices donde los elementos de las matrices de entrada se leen repetidamente. La opción `-sort=mem` agrupa los resultados por nivel de la jerarquía de memoria que sirvió el acceso.

El reporte resultante distribuye el porcentaje de accesos de carga entre los siguientes niveles de la jerarquía:

- **L1 hit**: acceso satisfecho por la caché de datos de nivel 1. Representa el caso ideal: latencia de 4–5 ciclos.
- **L2 hit**: acceso satisfecho por la caché de nivel 2. Latencia típica de 12–15 ciclos.
- **L3 hit** (LLC hit): acceso satisfecho por la caché de último nivel. Latencia típica de 30–40 ciclos según la arquitectura.
- **LFB hit** (*Line Fill Buffer*): acceso satisfecho por el buffer de relleno de línea, una estructura interna del procesador que almacena temporalmente líneas de caché en tránsito entre niveles. Un porcentaje alto de LFB hits indica alta presión de ancho de banda entre L1 y L2.

- **RAM hit:** acceso que no fue satisfecho por ningún nivel de caché y tuvo que recurrir a la memoria principal (DRAM). Latencia de 100–300 ciclos, dependiendo del acceso NUMA y la contención del bus de memoria.

La utilidad de `perf mem` en el contexto de la multiplicación de matrices reside en cuantificar la distribución efectiva de latencias: para $N = 128$ se espera que la mayoría de accesos sean L1 o L2 hits porque las matrices caben en los niveles superiores de caché, mientras que para $N = 1024$ el porcentaje de RAM hits debe escalar significativamente, ya que las tres matrices ocupan aproximadamente $3 \times 1024^2 \times 4 \approx 12$ MB, superando la capacidad típica de L2 (256 KB–1 MB) y presionando la LLC. Esta distribución complementa los miss rates de `perf stat` con información cualitativa sobre el *mix* de latencias que experimenta el programa en cada tamaño de problema.

La diferencia clave con `perf stat` es que `perf mem` proporciona una **vista por muestra y por nivel**, no solo contadores agregados. Mientras que `perf stat` indica cuántos fallos de L1 ocurrieron en total, `perf mem` indica qué fracción del tiempo el procesador estuvo esperando datos de L2, L3 o RAM, lo cual es directamente proporcional al impacto en el tiempo de ejecución.

3.4. `perf record`: muestreo basado en interrupciones de hardware

Mientras que `perf stat` reporta totales globales, `perf record` ofrece una vista espacial de dónde en el código ocurren los eventos de CPU. El script lo ejecuta con las opciones `-F 999` y `-g`:

- **-F 999:** configura la frecuencia de muestreo en aproximadamente 999 muestras por segundo. Cada muestra se captura mediante una interrupción de hardware generada por la PMU cuando el contador de ciclos alcanza un umbral predefinido.
- **-g:** captura el *call stack* completo en el momento de cada interrupción, lo que permite reconstruir no solo qué función estaba activa sino toda la cadena de llamadas que llegó hasta ella.

El resultado es un histograma que responde la pregunta: “¿qué fracción del tiempo de CPU estuvo el procesador ejecutando instrucciones de cada función?” Esta métrica es equivalente en concepto al flat profile de `gprof`, pero con dos ventajas clave: `perf record` no requiere recompilar el programa y mide el binario en sus condiciones reales de optimización `-O2`, incluyendo el código de las bibliotecas del sistema como la `libc`. Esto lo convierte en el perfilador de referencia para confirmar los resultados de `gprof` en condiciones de producción.

3.5. Valgrind Massif: perfilado de uso del heap

Massif es una herramienta del framework Valgrind que ejecuta el programa dentro de una máquina virtual de instrumentación binaria e intercepta todas las llamadas a las funciones de gestión de memoria dinámica (`malloc`, `calloc`, `realloc`, `free` y sus equivalentes en C++) sin necesidad de modificar el código fuente.

Durante la ejecución, Massif registra periódicamente *snapshots* del estado del heap, cada uno compuesto por:

- **Total bytes:** memoria total reservada en ese instante.
- **Useful heap bytes:** memoria correspondiente a bloques solicitados explícitamente por el programa.
- **Extra heap bytes:** overhead interno del gestor de memoria (metadata de los bloques, alineación y fragmentación interna).

La herramienta `ms_print` formatea estos snapshots en un reporte textual con una gráfica ASCII de la evolución temporal del heap, identificando el snapshot de pico y la pila de llamadas que originó las reservas de mayor tamaño.

El valor que el script extrae y almacena en el CSV es el **peak heap en MB**, que representa el máximo uso de memoria dinámica durante toda la ejecución. Para la multiplicación de matrices con representación *array de punteros* (`int **`), el modelo teórico de este valor es:

$$M_{\text{teórico}} = \frac{3 \cdot (N^2 \cdot 4 + N \cdot 8)}{1024^2} \text{ MB}, \quad (8)$$

donde $N^2 \cdot 4$ son los bytes de datos enteros de cada matriz y $N \cdot 8$ son los bytes del array de punteros a filas en una arquitectura de 64 bits. La diferencia entre el valor medido por Massif y este modelo teórico cuantifica el overhead de fragmentación interna del asignador de memoria del sistema.

3.6. Valgrind Cachegrind: simulación de la jerarquía de caché

Cachegrind es una herramienta del framework Valgrind que, a diferencia de `perf stat`, no utiliza contadores de hardware reales sino que **simula** la jerarquía de caché del procesador mediante instrumentación a nivel de instrucción (Nethercote & Seward, 2007). Cada operación de carga y almacenamiento del programa es interceptada y procesada por un modelo de caché configurable que reproduce la estructura de caché L1, L2 y LLC del sistema.

La métrica fundamental que produce Cachegrind es **Ir** (*Instruction References*): el conteo exacto y determinístico de instrucciones ejecutadas durante toda la corrida. A diferencia de los contadores de hardware, este valor es completamente reproducible entre ejecuciones, ya que no depende del estado transitorio del hardware.

El reporte generado por `cg_annotate` presenta tres niveles de granularidad:

- **Program totals:** instrucciones totales de todo el programa, incluyendo bibliotecas del sistema.
- **Function summary:** distribución porcentual de instrucciones por función, ordenadas de mayor a menor costo. Permite identificar el hotspot dominante con precisión absoluta.
- **Annotated source:** distribución de instrucciones línea por línea dentro del código fuente. Es el nivel más granular: identifica exactamente qué sentencias del código son las más costosas en términos de instrucciones ejecutadas, permitiendo correlacionar el costo con el acceso específico a elementos de las matrices.

La principal diferencia con `perf stat` es su naturaleza determinística: Cachegrind siempre produce el mismo conteo de instrucciones para una entrada dada, mientras que los contadores de hardware presentan variación estadística entre corridas por efectos del entorno. Sin embargo, Cachegrind ejecuta el programa entre 20 y 100 veces más lento que su velocidad normal debido al overhead de la instrumentación binaria, por lo que sus tiempos de ejecución no son representativos del rendimiento real.

3.7. Visión de conjunto y complementariedad

Las seis técnicas de perfilado del script cubren distintas dimensiones del comportamiento del programa, ninguna de ellas por sí sola es suficiente para construir un diagnóstico completo. La Tabla 1 resume la complementariedad entre herramientas.

Cuadro 1: Herramientas de perfilado utilizadas, su tipo de medición y su limitación principal

Herramienta	Tipo	Pregunta que responde	Limitación principal
Temporización	Wall-clock estático	¿Cuánto tarda? ¿Es estable?	Sin resolución por función
<code>gprof</code>	Instrumentación compilador	¿Qué función consume más CPU?	Binario modificado; resolución de 10 ms
<code>perf stat</code>	Contadores hardware (PMU)	¿Qué eventos de hardware ocurren?	Solo totales globales
<code>perf mem</code>	Muestreo PEBS de cargas	¿Desde qué nivel de memoria se sirven los accesos?	Solo procesadores con soporte PEBS
<code>perf record</code>	Muestreo por interrupciones	¿Dónde está el CPU la mayor parte del tiempo?	Resolución de ~ 1 ms
Valgrind Massif	Intercepción de heap	¿Cuánta memoria dinámica se usa?	5–30× overhead
Valgrind Cachegrind	Simulación de caché	¿Qué instrucciones son más costosas?	Modelo aproximado; 20–100× overhead

3.8. Resultados del perfilado

La Tabla 2 consolida los resultados de las seis técnicas de perfilado aplicadas a la multiplicación de matrices clásica para tamaños crecientes, $N \in \{128, 256, 512, 1024\}$. El uso de distintas herramientas permite identificar con precisión el origen de las limitaciones de rendimiento observadas. Algo que aclarar sobre el uso de distintos tamaños de matrices es que es útil para el perfilado de memoria y no para el de CPU sabiendo que con distintos tamaños de matrices se pueden identificar distintos cuellos de botella relacionados con la jerarquía de memoria, mientras que el perfilado de CPU se enfoca en identificar el hotspot dominante, que es el mismo para todos los tamaños. Por eso, aunque se ejecutaron las seis técnicas para los cuatro tamaños, el análisis del hotspot se centra exclusivamente en $N = 1024$, que es el caso más representativo del comportamiento real del programa en producción.

Nota: El perfilado se hizo en la maquina 1.

Cuadro 2: Caracterización de la implementación secuencial de multiplicación de matrices

N	Time mean (ms)	Std (ms)	GFLOPS	IPC	Cache Miss %	L1 Miss %	dTLB Miss %	Branch Miss %	Peak Heap (MB)
128	0.750	0.008	5.5924	5.6341	1.44	2.20	0.0000	0.0000	0.194
256	6.014	0.012	5.5794	3.4133	0.42	1.88	1.8833	0.0064	0.760
512	79.801	0.165	3.3638	2.8634	0.04	19.78	0.0035	0.1952	3.016
1024	702.284	5.780	3.0579	2.6175	1.81	29.74	0.0019	0.0956	12.027

3.8.1. Localización del hotspot

Las herramientas de perfilado coinciden en señalar la función de multiplicación como la única fuente relevante de cómputo. El reporte de gprof le atribuye el 100 % del tiempo de ejecución, mientras que perf record indica que para $N = 1024$ esta función concentra el 98.28 % de la actividad en el procesador, dejando apenas un 1.7 % restante a rutinas del programa como la inicialización y el llenado de matrices. Valgrind Cachegrind refuerza este resultado al asociarle el 98.5 % de las referencias de instrucción contabilizadas. Con esto podemos decir que todo el peso computacional recae sobre los tres bucles anidados del algoritmo, y cualquier optimización debe dirigirse exclusivamente a esa región.

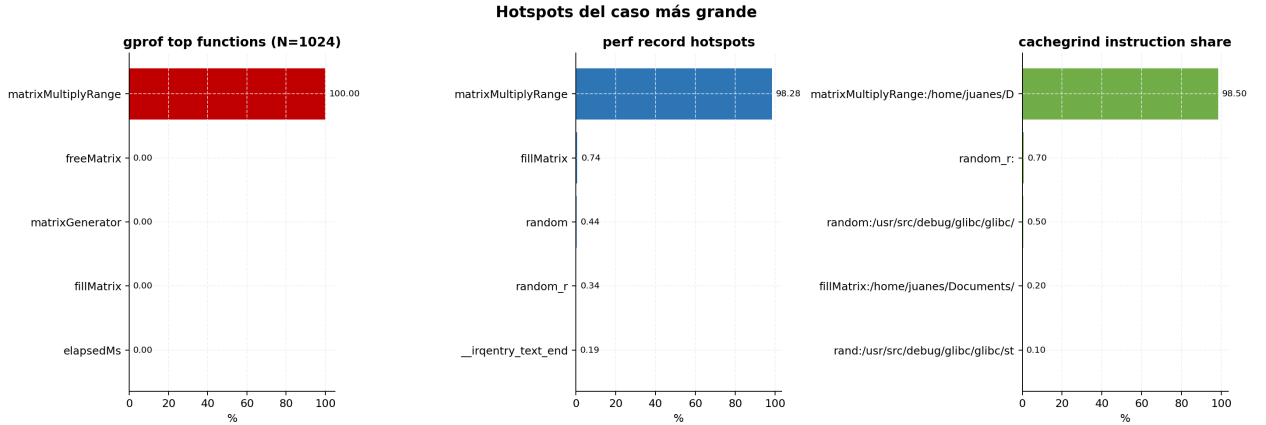


Figura 1: Distribución de tiempo e instrucciones en el hotspot principal durante el perfilado para el tamaño mayor ($N = 1024$).

3.8.2. Limitaciones de hardware

Las métricas recopiladas con perf stat muestran una caída sostenida del rendimiento a medida que crece el tamaño de la matriz, pasando de 5.53 GFLOPS para $N = 128$ hasta 3.07 GFLOPS para $N = 1024$.

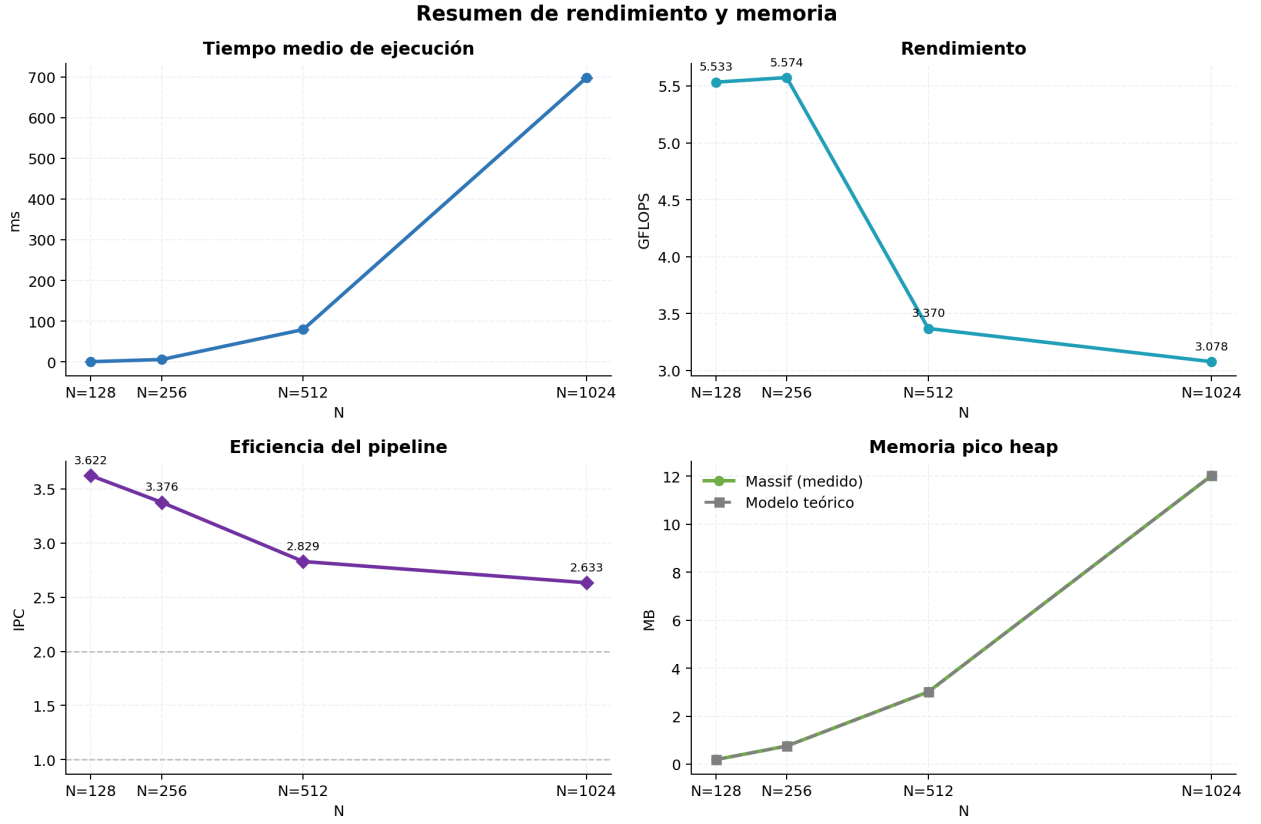


Figura 2: Resumen de rendimiento mostrando la caída progresiva de GFLOPS en función de los incrementos en tiempo y en la talla N .

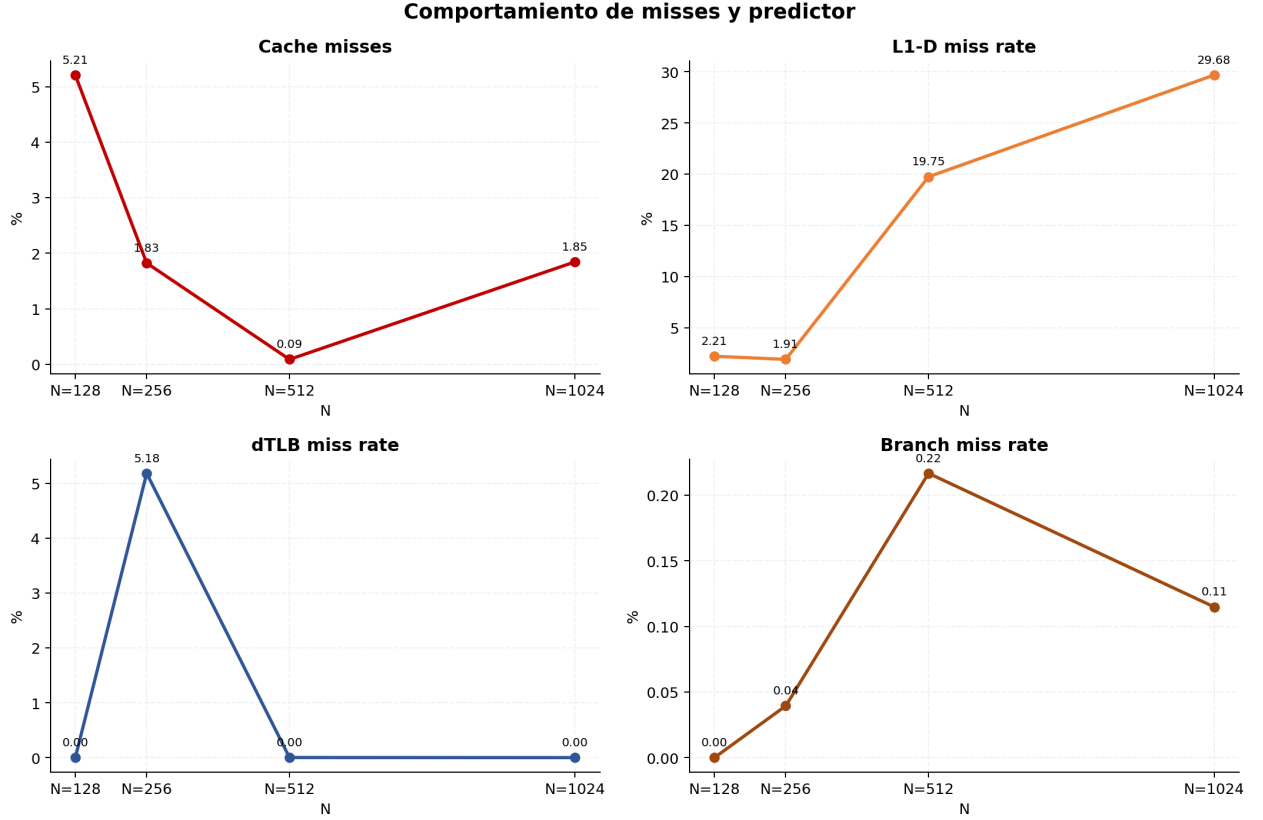


Figura 3: Correlación de tasas de fallo, acentuando el incremento en la caché de datos de nivel 1 frente a la estabilidad del predictor de ramas y la dTLB.

Esta degradación tiene tres causas que se pudieron indentificar. La primera es la presión sobre la caché de nivel 1: cuando las matrices son pequeñas ($N = 128$), los datos residen prácticamente en caché y la tasa de fallos en L1 es de apenas 2.21 %. Al escalar a $N = 1024$, esa tasa asciende a 29.68 %, lo que refleja un patrón de acceso desfavorable. En C, los arreglos bidimensionales se almacenan en orden de filas, de modo que recorrer la segunda matriz por columnas, como exige el producto interior, genera accesos no contiguos en memoria y produce líneas de caché ineficientes.

La segunda causa es la degradación del IPC. Para $N = 128$, el procesador logra un IPC de 5.63, aprovechando el paralelismo a nivel de instrucción disponible cuando los datos están en caché. A medida que los fallos de L1 obligan a recurrir a niveles más lentos de la jerarquía de memoria, el pipeline queda en espera y el IPC cae a 2.61 para $N = 1024$.

El tercer aspecto relevante es la estabilidad de los demás contadores. Los fallos de predicción de ramas se mantienen por debajo del 0.2 % en todos los casos, lo cual es esperable dado el flujo de control regular de los bucles. Las presiones sobre la TLB tampoco representan un factor limitante, con tasas cercanas a cero en todos los tamaños evaluados. Esto confirma que la caché de datos de nivel 1 es el cuello de botella dominante.

3.8.3. Uso de memoria dinámica

El análisis con Valgrind Massif muestra que el consumo de heap escala de forma cuadrática con N , pasando de 0.194 MB para $N = 128$ hasta 12.027 MB para $N = 1024$. Este comportamiento coincide con la estimación teórica esperada: tres matrices de enteros de 32 bits con $N = 1024$ requieren aproximadamente $1024 \times 1024 \times 4 \times 3 \approx 12$ MB. La ausencia de fragmentación o metadata excesiva confirma que el gestor de memoria del sistema no introduce sobrecarga anómala, y que cualquier limitante de rendimiento observada se origina en la ejecución del bucle de multiplicación y no en la gestión de memoria dinámica.

3.8.4. Distribución de accesos a memoria (perf mem / IBS)

La Figura 4 presenta la distribución de los accesos de carga entre los niveles de la jerarquía de memoria, obtenida mediante `perf mem` con el mecanismo IBS (*Instruction Based Sampling*) del procesador Ryzen 5 7600X. A diferencia de PEBS en arquitecturas Intel, IBS muestrea instrucciones en lugar de eventos de memoria directamente, lo que produce una fracción de muestras no clasificables reportadas como N/A, cuyo porcentaje se indica a la derecha de cada barra.

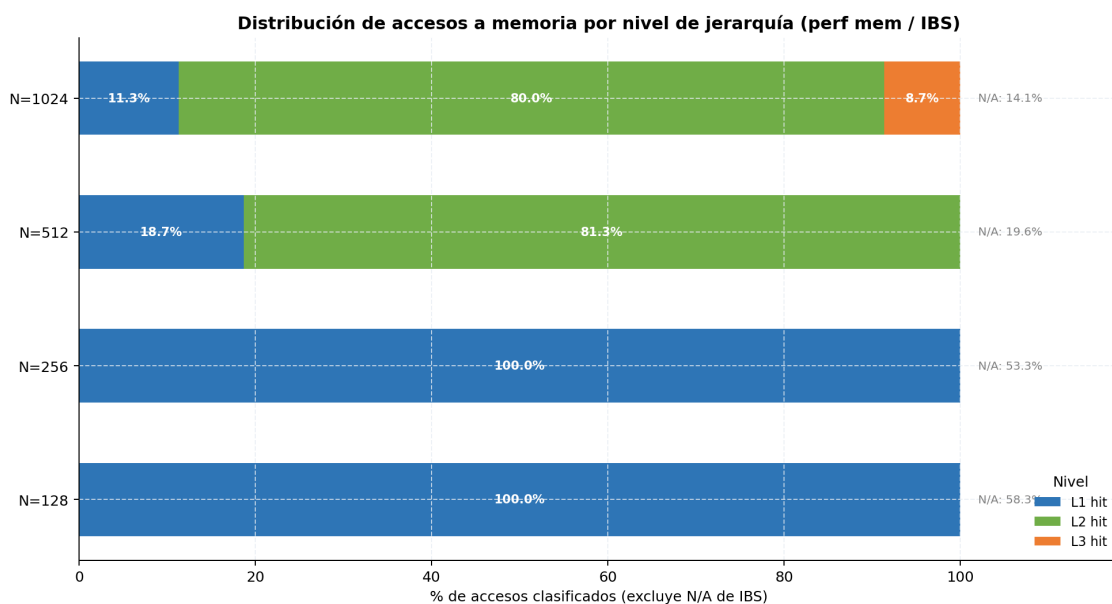


Figura 4: Distribución porcentual de accesos de carga clasificados por nivel de la jerarquía de memoria (`perf mem` / IBS). El porcentaje N/A indica muestras no clasificables por el mecanismo IBS de AMD Zen 4.

Para los tamaños pequeños ($N = 128$ y $N = 256$), la totalidad de los accesos clasificados se sirve desde L1, lo que refleja que las matrices caben activamente en caché durante la ejecución

y el hardware logra satisfacer la mayoría de las cargas sin escalar a niveles superiores. A partir de $N = 512$ el comportamiento cambia: el 81.3 % de los accesos clasificados provienen de L2, y L1 retiene apenas el 18.7 % restante. Esta transición indica que las matrices superan la capacidad efectiva de la caché L1 y el hardware debe recurrir al siguiente nivel de la jerarquía para servir la mayor parte de las cargas.

Para $N = 1024$, el patrón se consolida con un 80.0 % de accesos desde L2, un 11.3 % que aún se resuelve en L1 y un 8.7 % que requiere L3. La ausencia de accesos a RAM principal en los accesos clasificados, combinada con el 14.1 % de N/A, es consistente con el comportamiento de IBS en Zen 4: los accesos con latencias muy altas tienen mayor probabilidad de quedar sin clasificar al no completarse dentro de la ventana de muestreo. El resultado central es que el grueso de las cargas del algoritmo de multiplicación de matrices recae sobre L2 para matrices medianas y grandes, lo que concuerda con la tasa de L1 miss del 29.68 % reportada por `perf stat` para $N = 1024$. Los accesos que fallan en L1 son servidos mayoritariamente desde L2, lo que implica latencias de entre 5 y 12 ciclos adicionales por acceso. Dado que el bucle interno ejecuta n^2 accesos sobre la segunda matriz con un patrón de stride columnar, esta penalidad acumulada explica directamente la caída del IPC de 5.63 a 2.61 observada al escalar de $N = 128$ a $N = 1024$.

4. Benchmark

En esta sección se presentan los resultados del benchmark realizado sobre las dos máquinas descritas anteriormente. Para este benchmark se usó Geekbench 6.4.0, el cual se encargó de medir el rendimiento de ambas máquinas. Los resultados se pueden ver en los siguientes enlaces:

- <https://browser.geekbench.com/v6/cpu/17699747> (Máquina 1)
- <https://browser.geekbench.com/v6/cpu/17693088> (Máquina 2)

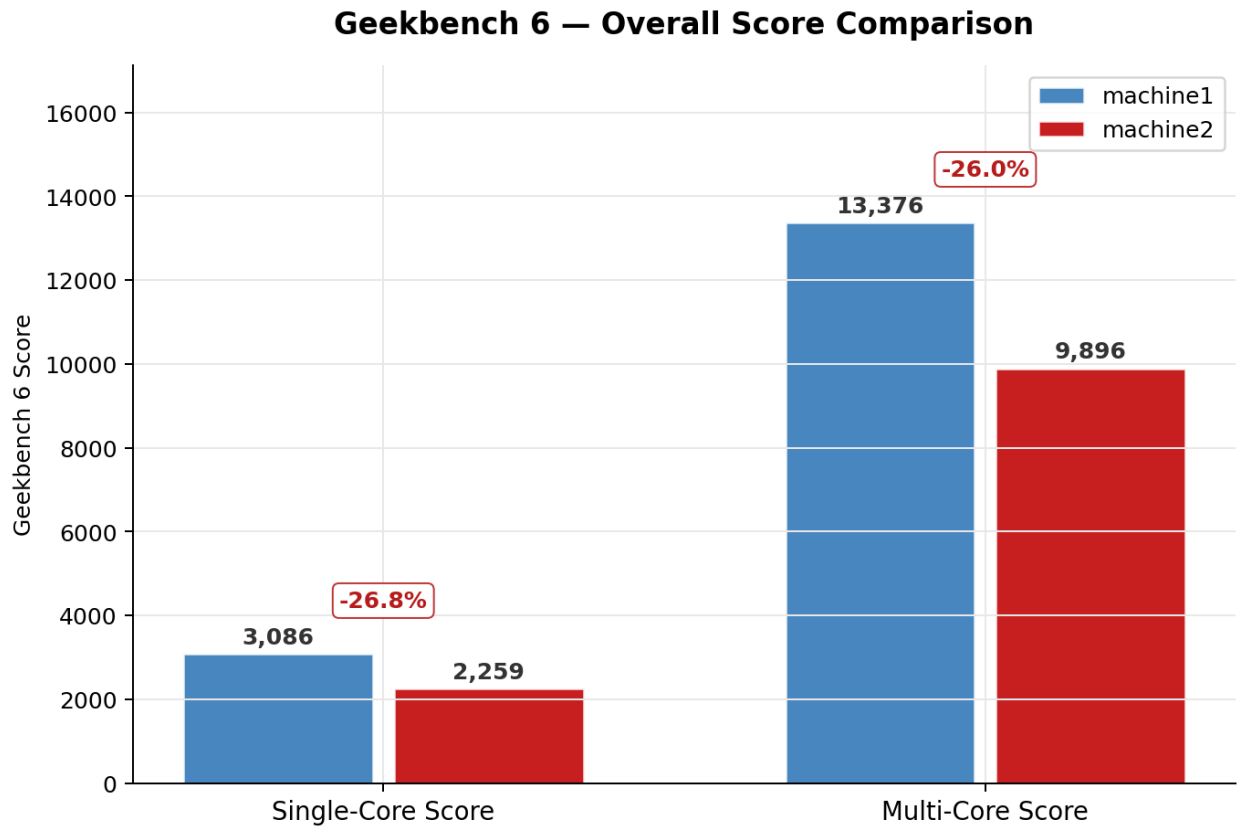


Figura 5: Score general de rendimiento para ambas máquinas

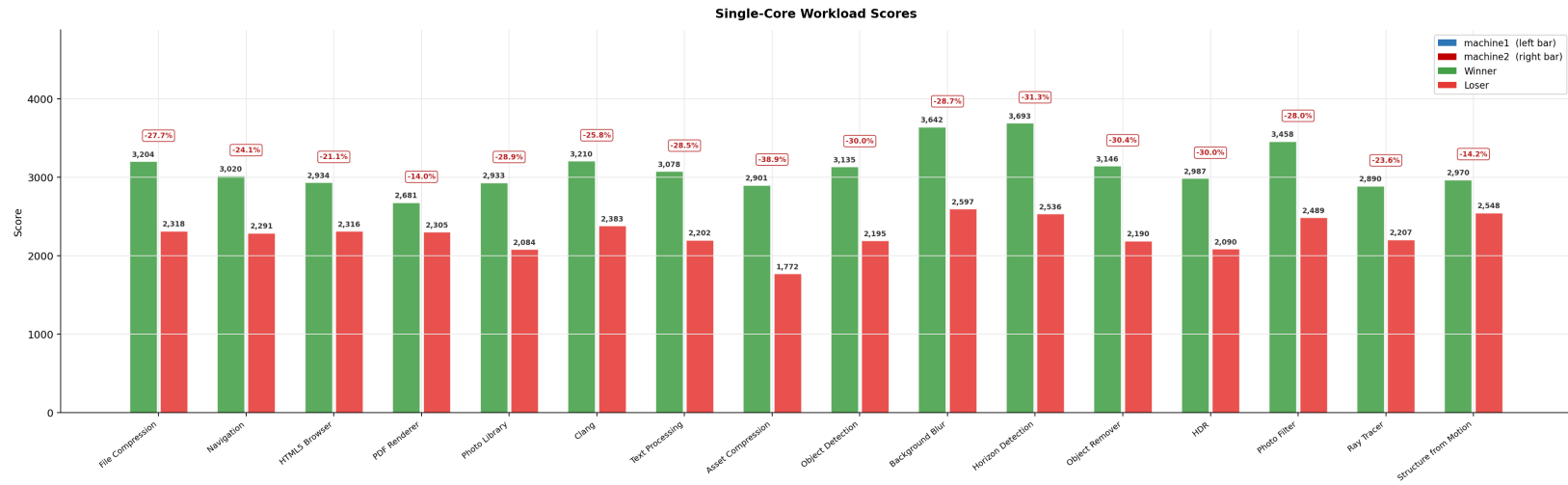


Figura 6: Rendimiento en single-core para ambas máquinas

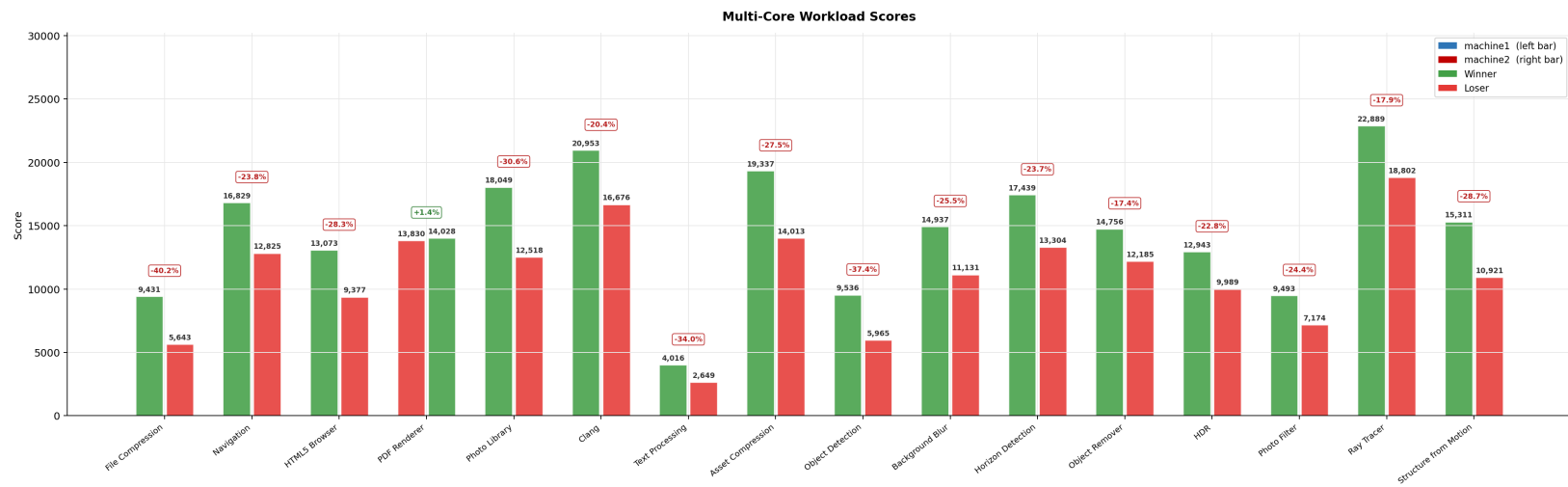


Figura 7: Rendimiento en multi-core para ambas máquinas

4.1. Descripción de los Sistemas Evaluados

Los experimentos fueron ejecutados sobre dos plataformas de cómputo con características arquitectónicas distintas. La primera máquina corresponde a un sistema de escritorio equipado con un procesador AMD Ryzen 5 7600X, basado en la microarquitectura Zen 4 (Family 25, Model 97, Stepping 2), con 6 núcleos físicos y 12 hilos de ejecución, operando a una frecuencia observada de 5.46 GHz bajo carga sostenida. Esta plataforma cuenta con 30.23 GB de memoria RAM y está montada sobre una tarjeta madre MSI PRO B650-S WIFI (chipset AMD B650), ejecutando Arch Linux como sistema operativo. La segunda máquina corresponde a un sistema portátil HP Victus 16, equipado con un procesador Intel Core i7-11800H, basado en la microarquitectura Tiger Lake-H (Family 6, Model 141, Stepping 1), con 8 núcleos físicos y 16 hilos de ejecución, operando a una frecuencia observada de 4.60 GHz. Esta plataforma dispone de 15.26 GB de memoria RAM y ejecuta Debian GNU/Linux 12 (Bookworm). Ambos sistemas soportan el conjunto de instrucciones AVX2, AVX-512 y VNNI, y fueron evaluados utilizando Geekbench 6.4.0 para Linux en su modalidad AVX2.

4.2. Resultados de Rendimiento

La evaluación de rendimiento mediante Geekbench 6.4.0 revela una ventaja sostenida del sistema desktop AMD Ryzen 5 7600X sobre el sistema portátil Intel Core i7-11800H en la totalidad de las métricas analizadas. En rendimiento de núcleo único, el Ryzen 5 7600X obtuvo una puntuación de 3086 frente a 2259 del i7-11800H, representando una ventaja del 26.8 % que refleja directamente la superioridad IPC de la microarquitectura Zen 4 respecto a Tiger Lake-H. En rendimiento multi-núcleo, el Ryzen 5 7600X alcanzó 13376 puntos frente a 9896 del i7-11800H, una diferencia del 26 %, resultado notable considerando que el procesador Intel dispone de dos núcleos físicos adicionales.

A nivel de subtests, el subtest de compresión de activos presentó la mayor brecha en núcleo único con un 40.2 % de ventaja para el Ryzen, atribuible al mayor aprovechamiento de instrucciones SIMD vectoriales. Por su parte, el subtest de procesamiento de texto exhibió el escalado multi-núcleo más bajo en ambas plataformas, 1.30x para el Ryzen y 1.20x para el Intel respecto al score de núcleo único, evidenciando dependencias seriales inherentes a esta carga de trabajo que limitan el beneficio del paralelismo. El subtest de trazado de rayos presentó el mayor ratio de escalado en ambos sistemas (aproximadamente 8x), consistente con su naturaleza paralela. Cabe señalar que las diferencias en configuración de memoria, política de energía y entorno térmico entre ambas plataformas constituyen variables de confusión que deben considerarse al interpretar los resultados comparativos.

5. Pruebas

En esta sección se presentan las pruebas realizadas para validar el funcionamiento del sistema desarrollado. Se describen los casos de prueba, los resultados obtenidos y las conclusiones derivadas de dichas pruebas. Se hace la comparación entre los resultados de las dos máquinas descritas anteriormente, con el fin de evaluar el rendimiento del sistema en diferentes entornos.

Para las optimizaciones por compilador se usaron las banderas `-O3 -Wall -march=native -funroll-loops -flto -ffast-math -fomit-frame-pointer`.

El significado de cada bandera es el siguiente:

- **-O3:** Esta bandera habilita un nivel de optimización agresivo, permitiendo al compilador aplicar una amplia gama de técnicas para mejorar el rendimiento del código.
- **-Wall:** Activa la mayoría de las advertencias del compilador, lo que ayuda a identificar posibles problemas en el código que podrían afectar su rendimiento o funcionalidad.
- **-march=native:** Permite al compilador generar código optimizado para la arquitectura específica del procesador en el que se está compilando, aprovechando las características y capacidades de hardware disponibles.
- **-funroll-loops:** Habilita el desenrollado de bucles, lo que puede mejorar el rendimiento al reducir la sobrecarga de control de bucle y aumentar la paralelización de las instrucciones.
- **-flto:** Habilita la optimización a nivel de enlace, lo que permite al compilador realizar optimizaciones globales en todo el programa, incluso entre diferentes archivos de código fuente.
- **-ffast-math:** Permite al compilador realizar optimizaciones agresivas en operaciones matemáticas, lo que puede mejorar el rendimiento a costa de la precisión en algunos casos como la evaluación de funciones trigonométricas.
- **-fomit-frame-pointer:** Indica al compilador que no utilice un puntero de marco para las funciones, es decir, que no cree un marco de pila para cada función, lo que puede mejorar el rendimiento al reducir la cantidad de instrucciones necesarias para gestionar la pila de llamadas.

5.1. Pruebas con OpenMP sin optimización por compilador

5.1.1. Pruebas con 2 hilos

machine1 (2 threads)					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	52,710	476,446	3.872,907	49.834,389	468.083,089
2	52,368	477,249	3.878,175	49.785,815	465.130,778
3	52,405	476,453	3.875,413	49.664,687	471.244,959
4	105,846	872,837	7.936,737	83.049,823	736.640,414
5	104,826	871,651	7.930,495	83.118,344	742.675,184
6	105,416	876,759	7.950,436	85.353,119	738.207,630
7	103,817	868,175	7.997,431	85.381,110	609.580,587
8	106,006	475,863	3.872,346	83.601,732	737.707,469
9	105,986	869,069	7.946,311	83.359,240	743.565,218
10	105,601	865,011	8.166,783	87.102,084	740.953,109
Promedio	89,498	712,951	6.342,703	74.025,034	645.378,844
Desv. Est.	24,233	193,081	2.016,129	15.929,295	122.163,534
CV (%)	27,08	27,08	31,79	21,52	18,93

Cuadro 3: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 2 hilos

5.1.2. Pruebas con 4 hilos

machine1 (4 threads)					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	52,683	435,342	3.972,652	41.338,015	367.526,215
2	52,202	431,382	3.958,917	41.294,067	314.280,066
3	52,681	432,606	3.985,667	41.419,009	316.482,124
4	51,714	431,940	3.962,978	41.220,366	315.856,441
5	52,713	431,420	3.969,467	41.374,154	317.546,886
6	52,111	433,488	3.964,539	41.355,428	316.041,259
7	51,591	431,365	3.965,847	41.067,030	366.010,610
8	51,920	431,581	3.956,449	40.526,926	366.109,846
9	51,993	434,153	3.962,092	41.101,329	366.055,692
10	49,501	442,685	3.954,117	40.896,489	330.950,776
Promedio	51,911	433,596	3.965,273	41.159,281	337.685,992
Desv. Est.	0,890	3,292	8,642	262,059	23.872,348
CV (%)	1,71	0,76	0,22	0,64	7,07

Cuadro 4: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 4 hilos

5.1.3. Pruebas con 6 hilos

machine1 (6 threads)					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	34,353	295,990	2.666,865	27.102,361	225.742,997
2	34,273	294,940	2.697,693	27.154,824	226.271,346
3	34,299	294,825	2.680,042	27.140,444	226.288,812
4	33,943	297,558	2.695,117	27.129,522	226.130,747
5	34,292	277,042	2.694,282	27.160,908	226.008,202
6	34,113	295,038	2.695,539	27.209,142	226.415,822
7	34,324	294,580	2.698,336	27.378,195	226.210,000
8	34,365	297,294	2.692,628	27.137,862	239.953,374
9	34,295	295,113	2.684,469	27.196,687	240.407,072
10	34,298	293,883	2.690,719	27.204,071	240.237,121
Promedio	34,255	293,626	2.689,569	27.181,402	230.366,549
Desv. Est.	0,123	5,638	9,351	73,405	6.440,075
CV (%)	0,36	1,92	0,35	0,27	2,80

Cuadro 5: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 6 hilos

5.1.4. Pruebas con 8 hilos

machine1 (8 threads)					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	24,034	202,552	1.876,715	19.146,828	167.753,892
2	24,174	204,324	1.869,716	19.074,945	172.227,315
3	23,614	206,152	1.869,526	19.465,286	167.621,693
4	23,945	197,717	1.875,406	19.448,636	169.148,746
5	24,112	197,878	1.870,942	19.448,722	167.026,430
6	24,236	206,722	1.872,806	19.562,146	168.885,813
7	24,264	198,601	1.868,560	19.404,398	171.266,467
8	23,579	198,132	1.868,898	19.571,730	172.608,639
9	24,162	205,551	1.894,854	19.553,356	168.093,107
10	24,045	207,082	1.873,434	19.640,924	171.368,745
Promedio	24,017	202,471	1.874,086	19.431,697	169.600,085
Desv. Est.	0,229	3,786	7,411	174,950	1.969,512
CV (%)	0,95	1,87	0,40	0,90	1,16

Cuadro 6: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 8 hilos

5.1.5. Pruebas con 12 hilos

machine1 (12 threads)					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	17,443	148,570	1.352,007	12.877,961	120.547,237
2	17,125	148,538	1.350,149	12.830,056	119.924,839
3	17,369	148,540	1.351,256	12.720,092	120.046,725
4	17,510	148,762	1.350,525	12.773,513	120.112,683
5	16,458	148,501	1.354,233	12.732,016	120.285,409
6	17,156	148,541	1.351,538	12.775,489	119.644,809
7	17,361	148,573	1.352,581	12.827,217	120.654,615
8	17,491	148,575	1.351,855	12.758,464	120.599,122
9	17,453	148,538	1.355,596	12.912,228	120.680,966
10	17,350	148,761	1.350,854	12.804,143	120.666,448
Promedio	17,272	148,590	1.352,059	12.801,118	120.316,285
Desv. Est.	0,298	0,088	1,610	58,614	349,655
CV (%)	1,73	0,06	0,12	0,46	0,29

Cuadro 7: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 12 hilos

5.1.6. Gráfica tiempo de ejecución

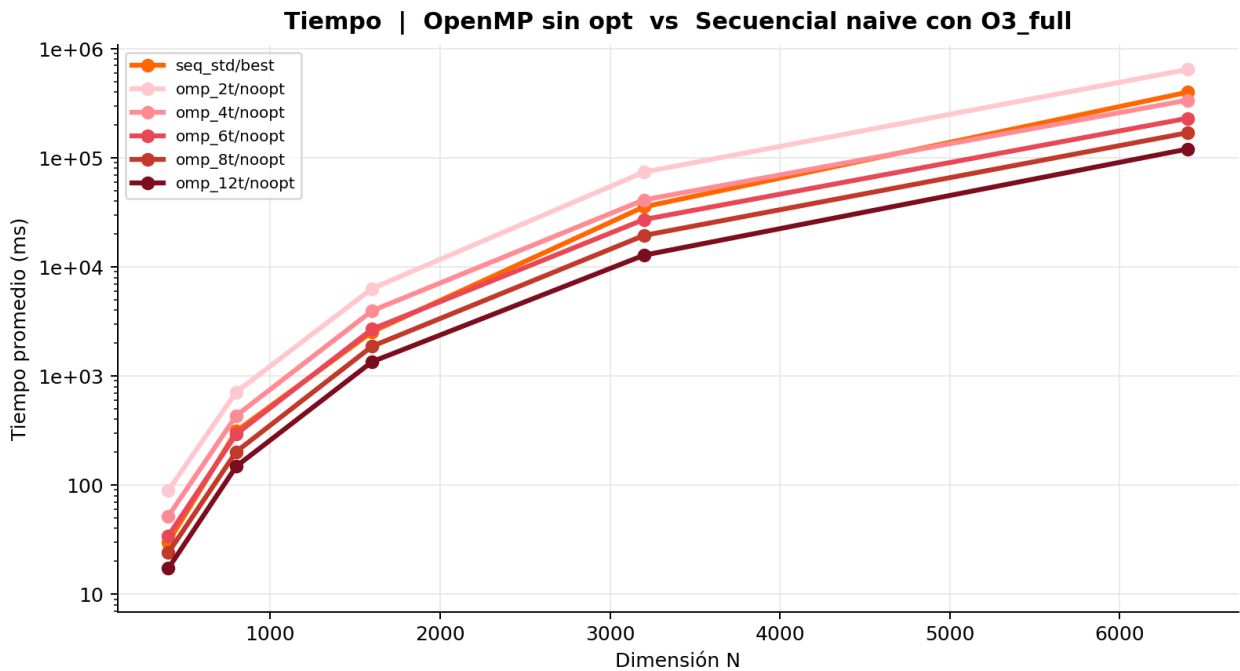


Figura 8: Gráfica de tiempo de ejecución para la multiplicación de matrices utilizando OpenMP sin optimización por compilador

5.1.7. Speedup con OpenMP sin optimización

7. OpenMP sin opt						
Implementacion	N=400	N=800	N=1,600	N=3,200	N=6,400	Sp. prom.
Secuencial naive con O3_full	30.1 ms (ref)	312.1 ms (ref)	2,543.1 ms (ref)	35,540.4 ms (ref)	399,073.0 ms (ref)	1.00x
OpenMP 2 hilos sin optimizar	89.5 ms 0.34x	713.0 ms 0.44x	6,342.7 ms 0.40x	74,025.0 ms 0.48x	645,378.8 ms 0.62x	0.45x
OpenMP 4 hilos sin optimizar	51.9 ms 0.58x	433.6 ms 0.72x	3,965.3 ms 0.64x	41,159.3 ms 0.86x	337,686.0 ms 1.18x	0.80x
OpenMP 6 hilos sin optimizar	34.3 ms 0.88x	293.6 ms 1.06x	2,689.6 ms 0.95x	27,181.4 ms 1.31x	230,366.5 ms 1.73x	1.19x
OpenMP 8 hilos sin optimizar	24.0 ms 1.25x	202.5 ms 1.54x	1,874.1 ms 1.36x	19,431.7 ms 1.83x	169,600.1 ms 2.35x	1.67x
OpenMP 12 hilos sin optimizar	17.3 ms 1.74x	148.6 ms 2.10x	1,352.1 ms 1.88x	12,801.1 ms 2.78x	120,316.3 ms 3.32x	2.36x

Cuadro 8: Resultados de Speedup para la multiplicación de matrices utilizando OpenMP

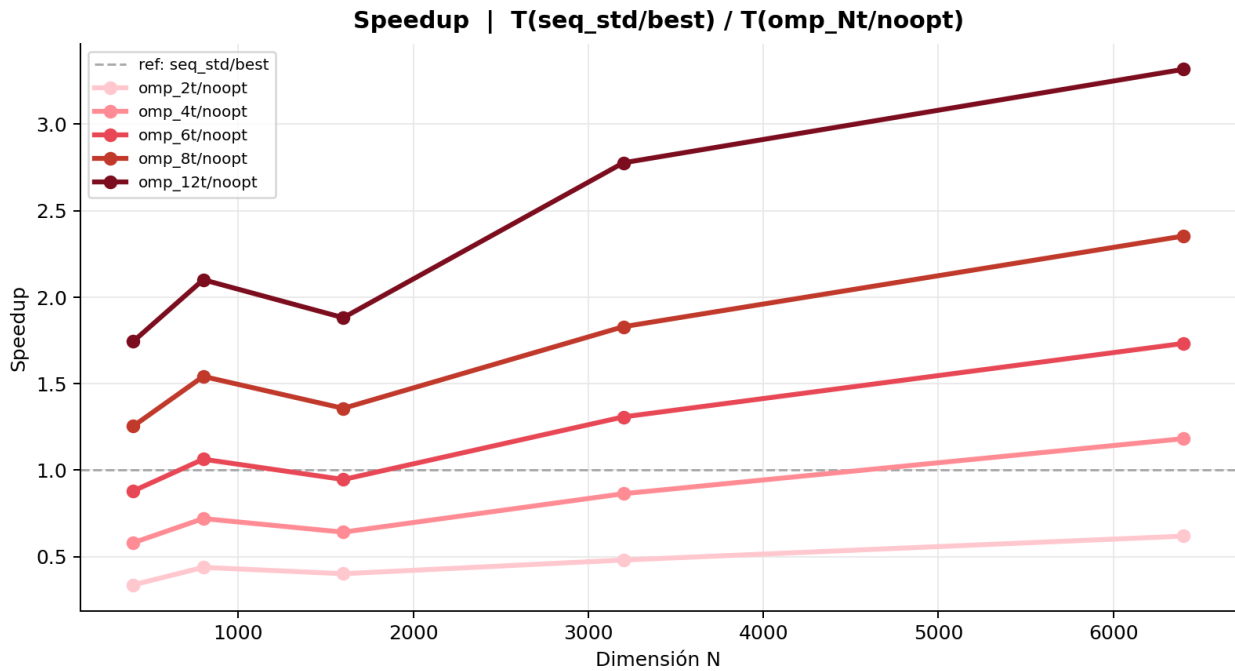


Figura 9: Gráfica de Speedup para la multiplicación de matrices utilizando OpenMP sin optimización

Como se ve en los resultados, entre más hilos se utilicen, el tiempo de ejecución disminuye, aunque no de manera lineal. Esto se debe a que la multiplicación de matrices es una tarea que se puede paralelizar, pero también tiene una parte secuencial que limita el speedup máximo que se puede alcanzar. Además, el overhead de crear y gestionar los hilos también

puede afectar el rendimiento, especialmente cuando se utilizan muchos hilos. En este caso, vemos como OpenMP con 12 hilos tiene un speedup de 2.36x en comparación con la versión secuencial, siendo el más rápido en estas pruebas.

5.2. Pruebas con OpenMP con optimización por compilador

5.2.1. Pruebas con 2 hilos

OpenMP 2 hilos con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	42,368	375,654	3.365,843	44.539,748	441.858,874
2	42,795	373,428	3.399,330	45.124,454	442.682,107
3	42,793	375,647	3.399,988	45.940,023	367.614,805
4	42,906	370,739	3.386,435	44.369,506	319.391,062
5	42,965	369,847	3.402,383	44.669,469	319.951,287
6	42,899	370,643	3.381,748	44.442,842	317.749,204
7	42,566	370,707	3.386,649	44.954,395	320.654,720
8	42,774	372,986	3.389,939	44.127,482	315.825,380
9	42,532	370,594	3.375,460	44.501,832	325.338,274
10	42,671	370,443	3.417,749	44.585,021	446.547,772
Promedio	42,727	372,069	3.390,552	44.725,477	361.761,348
Desv. Est.	0,180	2,092	14,121	485,621	55.492,056
CV (%)	0,42	0,56	0,42	1,09	15,34

Cuadro 9: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 2 hilos

5.2.2. Pruebas con 4 hilos

OpenMP 4 hilos con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	21,411	188,199	1.694,737	21.868,761	221.144,552
2	20,807	163,541	1.681,684	21.875,299	218.645,230
3	21,439	185,091	1.694,949	21.922,486	220.017,236
4	21,366	186,700	1.687,458	21.920,319	218.504,223
5	20,538	185,833	1.692,522	22.105,031	217.368,822
6	21,303	185,385	1.690,845	21.879,178	199.336,404
7	21,303	184,867	1.682,531	21.847,551	200.464,745
8	21,188	185,081	1.686,737	21.868,000	197.083,423
9	20,905	185,631	1.687,227	21.886,598	198.253,884
10	20,686	188,193	1.685,957	21.901,648	198.908,991
Promedio	21,095	183,852	1.688,465	21.907,487	208.972,751
Desv. Est.	0,314	6,870	4,433	69,550	10.236,250
CV (%)	1,49	3,74	0,26	0,32	4,90

Cuadro 10: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 4 hilos

5.2.3. Pruebas con 6 hilos

OpenMP 6 hilos con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	14,205	124,638	1.133,425	14.630,997	135.065,333
2	14,349	124,717	1.133,236	14.628,084	135.031,467
3	14,205	124,760	1.126,336	14.606,231	141.699,599
4	14,374	124,831	1.130,410	14.540,141	140.334,752
5	14,370	124,633	1.131,885	14.619,816	136.563,409
6	14,366	124,712	1.128,999	14.619,765	134.982,496
7	14,316	124,716	1.125,627	14.586,914	136.343,556
8	14,354	124,773	1.138,768	14.657,505	134.439,221
9	14,204	124,652	1.129,223	14.673,147	136.131,646
10	14,370	124,645	1.128,906	14.624,914	133.763,292
Promedio	14,311	124,708	1.130,682	14.618,751	136.435,477
Desv. Est.	0,072	0,063	3,654	34,768	2.451,892
CV (%)	0,50	0,05	0,32	0,24	1,80

Cuadro 11: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 6 hilos

5.2.4. Pruebas con 8 hilos

OpenMP 8 hilos con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	10,625	93,484	839,759	10.875,106	105.129,709
2	10,616	89,500	843,527	10.852,801	104.179,469
3	10,614	92,570	839,659	10.826,405	106.173,603
4	10,799	93,156	839,728	10.914,293	106.275,813
5	10,693	92,893	842,158	10.871,770	105.254,113
6	10,462	94,451	841,843	10.907,212	106.706,482
7	10,624	94,160	841,233	10.972,839	104.604,647
8	10,749	91,243	842,288	10.838,175	106.244,555
9	10,663	92,661	840,533	10.829,220	106.725,565
10	10,691	92,691	843,585	10.854,596	106.735,484
Promedio	10,654	92,681	841,431	10.874,242	105.802,944
Desv. Est.	0,086	1,358	1,419	43,453	889,905
CV (%)	0,81	1,46	0,17	0,40	0,84

Cuadro 12: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 8 hilos

5.2.5. Pruebas con 12 hilos

OpenMP 12 hilos con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	7,316	62,489	564,903	6.734,835	68.260,682
2	7,286	62,479	564,923	6.721,784	68.969,214
3	7,272	62,498	565,399	6.748,110	68.560,270
4	7,327	62,501	565,028	6.736,491	68.834,321
5	7,298	62,489	565,646	6.741,619	69.801,397
6	7,269	62,654	564,875	6.743,477	69.040,134
7	7,296	62,459	564,856	6.773,796	68.360,397
8	7,328	62,486	564,900	6.652,650	68.706,412
9	7,316	62,487	564,872	6.722,551	68.541,395
10	7,301	62,484	564,746	6.723,406	69.279,754
Promedio	7,301	62,503	565,015	6.729,872	68.835,398
Desv. Est.	0,020	0,052	0,268	29,627	438,763
CV (%)	0,27	0,08	0,05	0,44	0,64

Cuadro 13: Resultados de tiempo para la multiplicación de matrices utilizando OpenMP con 12 hilos

5.2.6. Gráfica de tiempo

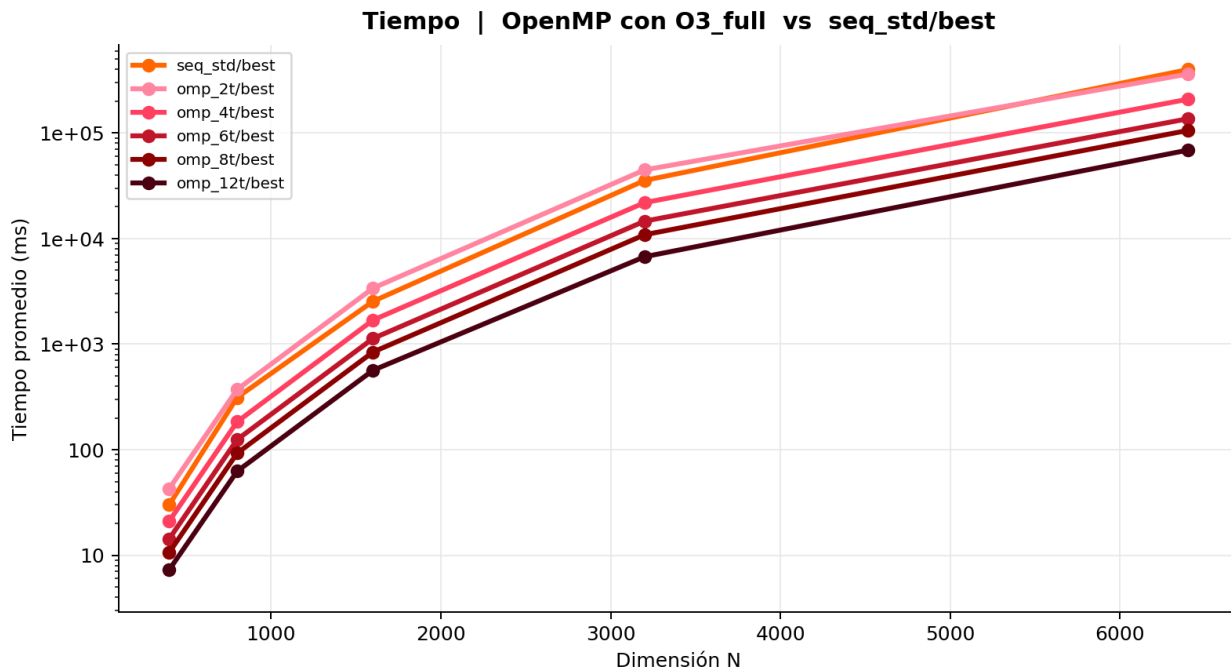


Figura 10: Gráfica de tiempo para la multiplicación de matrices utilizando OpenMP con optimización por compilador

5.2.7. Speedup con OpenMP con optimización

8. OpenMP con opt						
Implementacion	N=400	N=800	N=1,600	N=3,200	N=6,400	Sp. prom.
Secuencial naive con O3_full	30.1 ms (ref)	312.1 ms (ref)	2,543.1 ms (ref)	35,540.4 ms (ref)	399,073.0 ms (ref)	1.00x
OpenMP 2 hilos con O3_full	42.7 ms 0.70x	372.1 ms 0.84x	3,390.6 ms 0.75x	44,725.5 ms 0.79x	361,761.3 ms 1.10x	0.84x
OpenMP 4 hilos con O3_full	21.1 ms 1.43x	183.9 ms 1.70x	1,688.5 ms 1.51x	21,907.5 ms 1.62x	208,972.8 ms 1.91x	1.63x
OpenMP 6 hilos con O3_full	14.3 ms 2.10x	124.7 ms 2.50x	1,130.7 ms 2.25x	14,618.8 ms 2.43x	136,435.5 ms 2.92x	2.44x
OpenMP 8 hilos con O3_full	10.7 ms 2.83x	92.7 ms 3.37x	841.4 ms 3.02x	10,874.2 ms 3.27x	105,802.9 ms 3.77x	3.25x
OpenMP 12 hilos con O3_full	7.3 ms 4.13x	62.5 ms 4.99x	565.0 ms 4.50x	6,729.9 ms 5.28x	68,835.4 ms 5.80x	4.94x

Cuadro 14: Resultados de Speedup para la multiplicación de matrices utilizando OpenMP con optimización por compilador

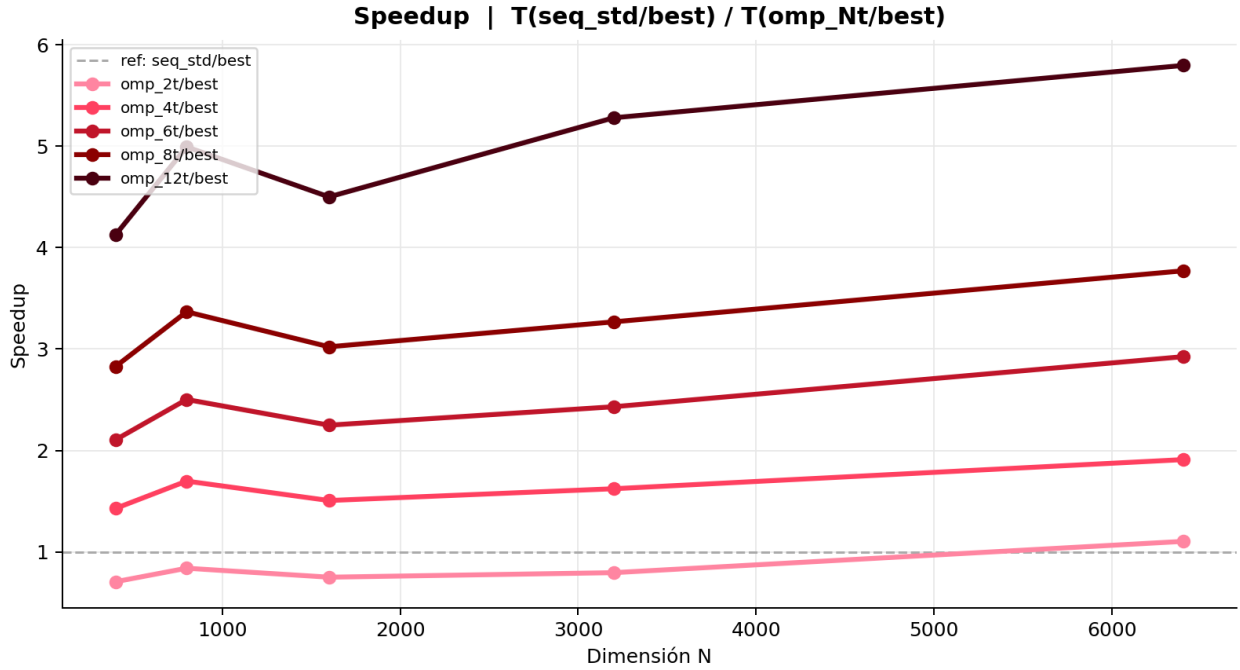


Figura 11: Gráfica de Speedup para la multiplicación de matrices utilizando OpenMP con optimización

En este caso, vemos una mejora significativa en el tiempo de ejecución al utilizar la optimización por compilador, especialmente a medida que se incrementa el número de hilos. El speedup también muestra una mejora considerable, alcanzando un máximo de 4.94x con 12 hilos, lo que indica que la optimización por compilador ha permitido aprovechar mejor los recursos del sistema y reducir el overhead asociado con la paralelización.

5.3. Comparación Pthreads vs OpenMP

Haciendo las comparaciones entre los resultados obtenidos en el Caso de Estudio 1 donde se usó Pthreads y los resultados obtenidos en el Caso de Estudio 2 donde se usó OpenMP, se puede observar que ambos enfoques presentan mejoras significativas en el tiempo de ejecución a medida que se incrementa el número de hilos. Sin embargo, OpenMP tiende a mostrar un mejor rendimiento en comparación con Pthreads, especialmente cuando se utilizan más hilos. Esto se debe a que OpenMP está diseñado para facilitar la paralelización y optimización automática, mientras que Pthreads requiere una gestión manual de los hilos y la sincronización, lo que puede introducir sobrecarga adicional. En general, OpenMP es una opción más eficiente y fácil de usar para la multiplicación de matrices en entornos de programación paralela. En esta comparación se usaron los mejores resultados obtenidos para cada enfoque, tanto para Pthreads como para OpenMP con optimización por compilador como sin optimización por

compilador.

5.3.1. Resultados sin optimización por compilador

8. OpenMP vs pthreads						
Implementacion	N=400	N=800	N=1,600	N=3,200	N=6,400	Sp. prom.
Secuencial naive con O3_full	30.1 ms (ref)	312.1 ms (ref)	2,543.1 ms (ref)	35,540.4 ms (ref)	399,073.0 ms (ref)	1.00x
Concurrencia 6 hilos sin optimizar	27.1 ms 1.11x	231.5 ms 1.35x	1,901.8 ms 1.34x	19,903.1 ms 1.79x	183,628.2 ms 2.17x	1.55x
Concurrencia 12 hilos sin optimizar	27.3 ms 1.10x	241.3 ms 1.29x	1,906.2 ms 1.33x	20,080.9 ms 1.77x	195,798.8 ms 2.04x	1.51x
OpenMP 12 hilos sin optimizar	17.3 ms 1.74x	148.6 ms 2.10x	1,352.1 ms 1.88x	12,801.1 ms 2.78x	120,316.3 ms 3.32x	2.36x

Cuadro 15: Comparación de tiempo de ejecución entre OpenMP y Pthreads para la multiplicación de matrices

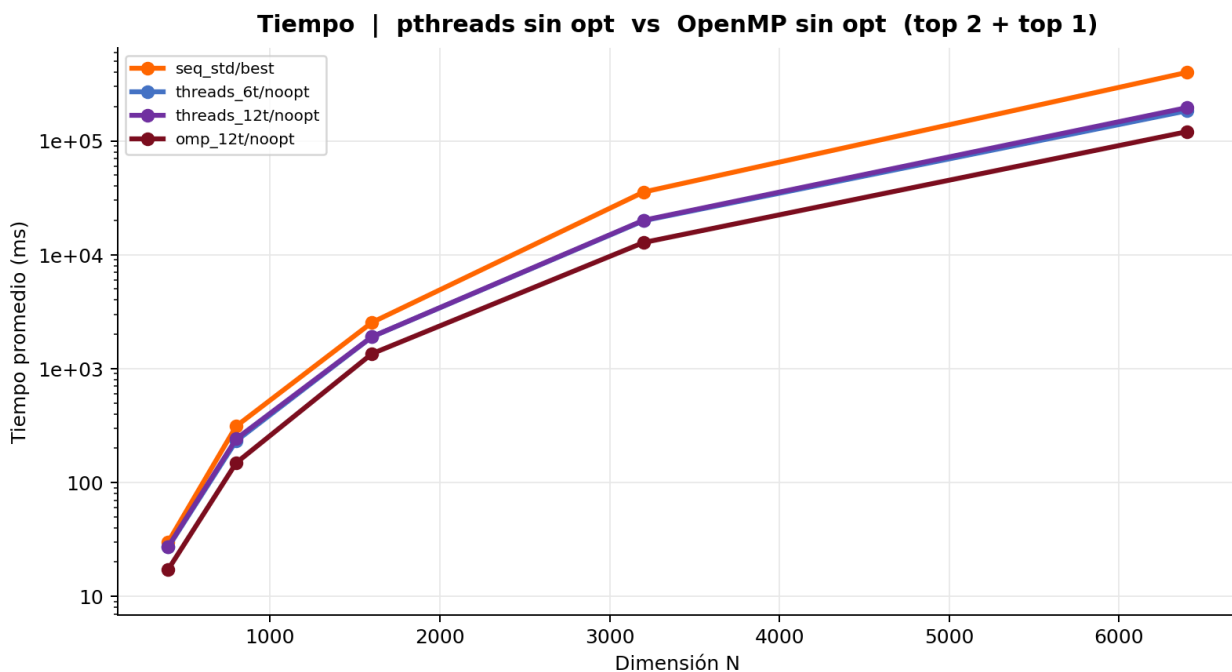


Figura 12: Gráfica de comparación de tiempo de ejecución entre OpenMP y Pthreads para la multiplicación de matrices

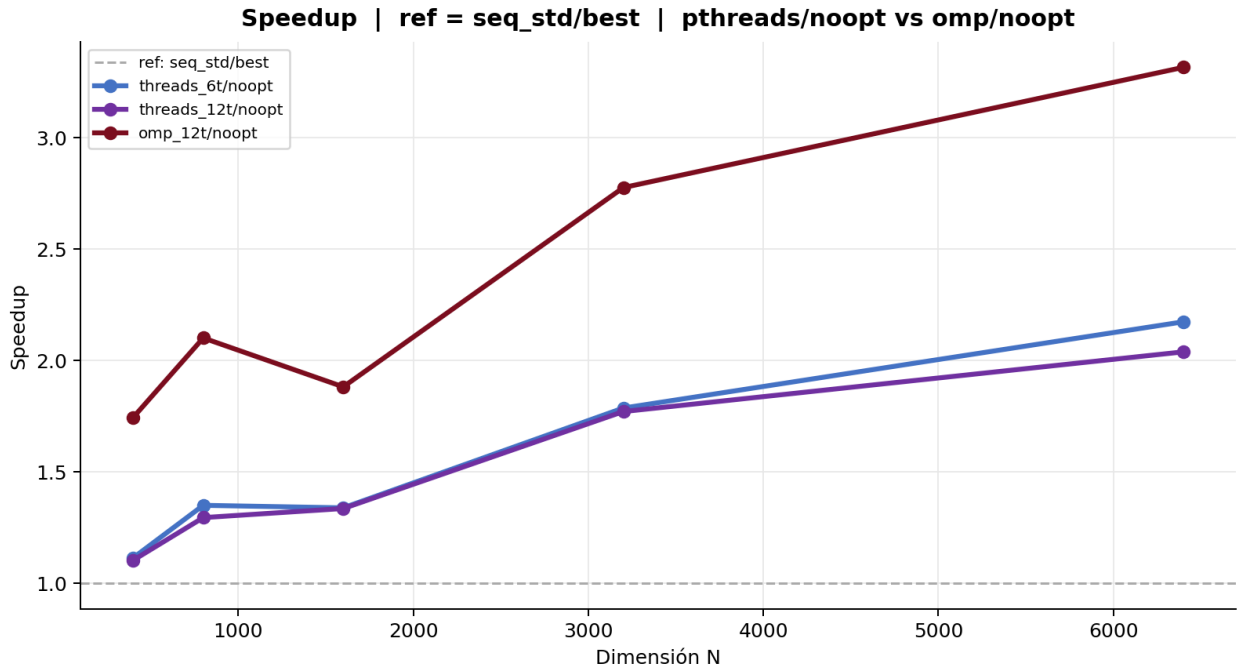


Figura 13: Gráfica de comparación de Speedup entre OpenMP y Pthreads para la multiplicación de matrices

5.3.2. Resultados con optimización por compilador

10. pthreads vs OMP con opt						
Implementacion	N=400	N=800	N=1,600	N=3,200	N=6,400	Sp. prom.
Secuencial naive con O3_full	30.1 ms (ref)	312.1 ms (ref)	2,543.1 ms (ref)	35,540.4 ms (ref)	399,073.0 ms (ref)	1.00x
Concurrencia 6 hilos con O3_full	5.4 ms 5.62x	52.8 ms 5.91x	429.8 ms 5.92x	6,036.0 ms 5.89x	82,004.9 ms 4.87x	5.64x
Concurrencia 12 hilos con O3_full	5.7 ms 5.26x	55.6 ms 5.61x	436.7 ms 5.82x	6,106.2 ms 5.82x	82,384.7 ms 4.84x	5.47x
OpenMP 12 hilos con O3_full	7.3 ms 4.13x	62.5 ms 4.99x	565.0 ms 4.50x	6,729.9 ms 5.28x	68,835.4 ms 5.80x	4.94x

Cuadro 16: Comparación de tiempo de ejecución entre OpenMP y Pthreads para la multiplicación de matrices

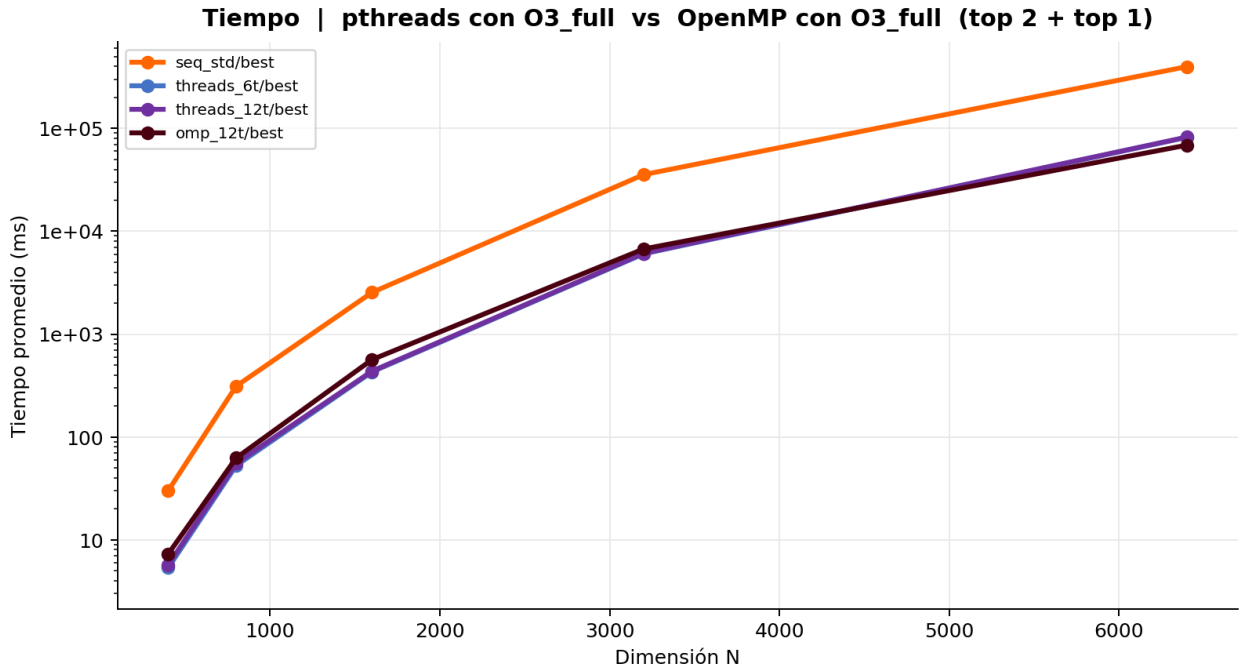


Figura 14: Gráfica de comparación de tiempo de ejecución entre OpenMP y Pthreads para la multiplicación de matrices

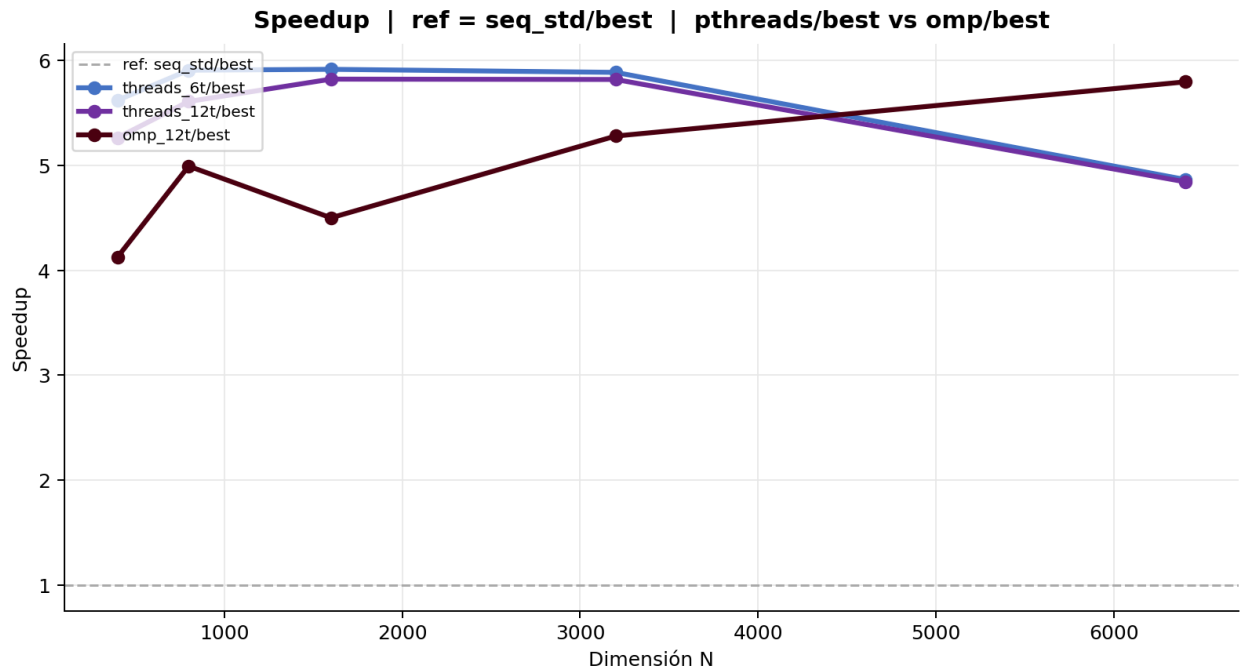


Figura 15: Gráfica de comparación de Speedup entre OpenMP y Pthreads para la multiplicación de matrices

Aquí se observa algo interesante, la version con Pthreads optmizada por compilador tiene un mejor que la version con OpenMP optimizada por compilador en los tamaños más pequeños, pero a medida que el tamaño de las matrices aumenta, OpenMP con optimización por compilador supera a Pthreads con optimización por compilador, mostrando un mejor rendimiento y speedup. Esto puede deberse a que OpenMP está diseñado para aprovechar mejor los recursos del sistema y reducir el overhead asociado con la paralelización, mientras que Pthreads puede tener limitaciones en la gestión de hilos y sincronización a medida que el tamaño del problema aumenta.

6. Conclusiones

1. Con 2 hilos y sin optimización, OpenMP solo alcanza un speedup promedio de 0.45x respecto a la versión secuelcial optimizada, es decir, es más del doble de lento. Con 4 hilos llega a 0.80x y recién a partir de 6 hilos (0.88x) empieza a acercarse a 1x. La causa es que el código paralelo de OpenMP sin optimización parte de un baseline serial lento, sin vectorización, y el paralelismo necesita varios hilos para compensar esa desventaja inicial.
2. Los resultados muestran que los 6 hilos con Pthreads con optimización alcanza un speedup de 5.64x y la versión de 12 hilos de 5.47x, ambos por encima del mejor resultado de OpenMP (4.94x con 12 hilos). Una explicación posible es que las regiones paralelas de OpenMP introducen barreras implícitas y posibles problemas de aliasing de punteros que el compilador no puede resolver completamente, limitando la vectorización interna de cada hilo. En pthreads, el compilador tiene control total sobre cada función de hilo, lo que permite un uso más agresivo de AVX2.
3. Con Pthreads, tanto sin optimización como con optimización, 6 hilos superan a 12 hilos. Esto es coherente con la arquitectura del Ryzen 7600X, que cuenta con 6 núcleos físicos y 6 hilos SMT. En una carga fuertemente compute-bound como la multiplicación de matrices, el SMT compite por las unidades de punto flotante del mismo núcleo y no aporta beneficios. En cambio, con OpenMP, 12 hilos optimizados (4.94x) sí superan a 6 hilos optimizados (2.44x), lo que sugiere que el runtime de OpenMP maneja la afinidad o la distribución del trabajo de manera que logra extraer algún beneficio del SMT, aunque con menor eficiencia por hilo.
4. La versión de 6 hilos de Pthreads con optimización, la eficiencia es $5,64/6 = 0,94$, prácticamente una escalabilidad lineal perfecta. En La versión de 12 hilos de Pthreads

con optimización, la eficiencia cae a $5,47/12 = 0,46$, reflejando el impacto negativo del SMT. En la version de OpenMP con 12 hilos, la eficiencia es $4,94/12 = 0,41$, similar a pthreads con 12 hilos pero partiendo de un speedup menor. En la versión de OpenMP sin optimización, la eficiencia cae a $2,36/12 = 0,20$, lo que muestra que el overhead domina cuando el código serial es lento. El valor de 0.94 en la versión de Pthreads con 6 hilos optimizados es el dato más llamativo del benchmark y confirma que, a partir de 6 hilos, el cuello de botella principal es el hardware y no la implementación.

5. Para $N = 6400$ las tres matrices pesan aproximadamente 491MB, completamente fuera del alcance de cualquier caché en ambas máquinas. El problema viene del orden en que el algoritmo recorre los datos: en lugar de leer posiciones de memoria consecutivas para aprovechar la cacheline, salta de un extremo al otro de la matriz en cada paso. Eso hace imposible que el procesador anticipe qué dato va a necesitar a continuación, así que cada vez que lo requiere tiene que ir a buscarlo hasta la memoria principal, deteniéndose a esperar mientras el dato llega. Ese tiempo de espera es precisamente lo que los datos de perfilado muestran: la eficiencia del procesador, medida como instrucciones ejecutadas por ciclo, cae de 3.62 con $N = 128$, donde todo el problema cabe cerca del procesador, a 2.63 con $N = 1024$, cuando los datos ya no caben en los niveles más rápidos. Con $N = 6400$ esa cifra sería aún menor, porque prácticamente cada acceso implica un viaje a la memoria principal.
6. La optimización de compilador beneficia más a pthreads que a OpenMP. Pongamos el ejemplo con 6 hilos: al pasar de sin optimización a con optimización, la implementación con pthreads de 6 hilos multiplica su speedup por 3.63x, pasando de 1.55x a 5.64x. En contraste, OpenMP con 6 hilos solo lo multiplica por 2.05x, de 1.19x a 2.44x, aun si lo comparamos con OpenMP con 12 hilos multiplica su speedup por 2.09x, de 2.36x a 4.94x. La diferencia es significativa y se explica porque el compilador tiene plena libertad para vectorizar las funciones de cada hilo en pthreads, al ser funciones C normales y aisladas. En cambio, dentro de una región paralela `#pragma omp parallel for`, el compilador debe asumir aliasing de punteros entre hilos y la presencia de barreras implícitas de sincronización, lo que inhibe parte de la vectorización automática.
7. Con optimización de compilador, pthreads con 6 hilos alcanza un speedup de 5.64x, mientras que OpenMP con 6 hilos solo llega a 2.44x usando los mismos recursos de hardware. Esto implica que pthreads es aproximadamente 2.3 veces más rápido. Esta diferencia no puede atribuirse únicamente al overhead de sincronización, que con 6 hilos es bajo, sino principalmente a las restricciones de vectorización presentes en OpenMP.

Este caso actúa como el experimento de control más limpio del benchmark, ya que elimina los efectos del SMT y del uso de conteos de hilos desiguales.

8. OpenMP con optimización escala casi linealmente al aumentar el número de hilos. Dentro de la configuración de OpenMP con optimización, el speedup entre conteos consecutivos de hilos sigue una proporción cercana al ideal: de 2 a 4 hilos la mejora es 1.95x frente al ideal de 2x; de 4 a 6 hilos es 1.50x frente a 1.5x ideal; de 6 a 8 hilos es 1.33x frente a 1.33x ideal; y de 8 a 12 hilos es 1.52x frente a 1.5x ideal. Esta escalabilidad casi lineal se mantiene hasta 12 hilos, incluyendo el uso de SMT, y contrasta con pthreads, donde la ganancia colapsa al pasar de 6 a 12 hilos.
9. El speedup aumenta con el tamaño del problema en todas las variantes paralelas. En OpenMP con 12 hilos y optimización, el speedup crece desde 4.13x en $N = 400$ hasta 5.80x en $N = 6400$. Este patrón se repite en todas las configuraciones evaluadas. La razón es que el overhead de gestión de hilos es aproximadamente constante en tiempo absoluto, mientras que el trabajo útil crece como $O(N^3)$. A medida que el problema escala, la fracción paralela domina cada vez más el tiempo total de ejecución. Como consecuencia, los speedups promediados entre tamaños subestiman el rendimiento real en los problemas grandes, que son los más relevantes en la práctica.
10. En pthreads sin optimización, pasar de 6 a 12 hilos produce una ligera caída de rendimiento, de 1.55x a 1.51x. En OpenMP sin optimización, en cambio, pasar de 8 a 12 hilos genera una ganancia real, de 1.67x a 2.36x. Esto sugiere que el runtime de OpenMP distribuye el trabajo y gestiona la afinidad de hilos de forma más eficiente que una implementación manual con pthreads cuando se entra en territorio SMT con código no vectorizado. No obstante, esta ventaja desaparece completamente cuando se habilitan las optimizaciones del compilador.

Referencias

- Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming* (2nd). Addison-Wesley.
- Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd). The MIT Press.
- Dagum, L., & Menon, R. (1998). OpenMP: An Industry-Standard API for Shared-Memory Programming. *IEEE Computational Science and Engineering*, 5(1), 46-55. <https://doi.org/10.1109/99.660313>
- Dongarra, J. J., Du Croz, J., Duff, I. S., & Hammarling, S. (1990). A Set of Level 3 Basic Linear Algebra Subprograms. *ACM Transactions on Mathematical Software*, 16(1), 1-17. <https://doi.org/10.1145/77626.79170>
- Drepper, U. (2007). *What Every Programmer Should Know About Memory* (Technical Report). Red Hat, Inc. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- Graham, S. L., Kessler, P. B., & McKusick, M. K. (1982a). gprof: A Call Graph Execution Profiler. *SIGPLAN Notices*, 17(6), 120-126. <https://doi.org/10.1145/872726.806987>
- Graham, S. L., Kessler, P. B., & McKusick, M. K. (1982b). gprof: A Call Graph Execution Profiler. *Proceedings of the ACM SIGPLAN '82 Symposium on Compiler Construction*, 17(6), 120-126. <https://doi.org/10.1145/800230.806987>
- Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing* (2nd). Addison-Wesley.
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- Lam, M. S., Rothberg, E. E., & Wolf, M. E. (1991). The Cache Performance and Optimizations of Blocked Algorithms. *ACM SIGARCH Computer Architecture News*, 19(2), 63-74. <https://doi.org/10.1145/115953.115963>
- Nethercote, N., & Seward, J. (2007). Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation. *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 89-100. <https://doi.org/10.1145/1250734.1250746>
- Patterson, D. A., & Hennessy, J. L. (2017). *Computer Architecture: A Quantitative Approach* (6th). Morgan Kaufmann.

- Primate Labs Inc. (2024, marzo). *Geekbench 6 Benchmark Internals* (inf. téc.). Primate Labs.
<https://www.geekbench.com/doc/geekbench6-benchmark-internals.pdf>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th). Wiley.
- Strang, G. (2016). *Introduction to Linear Algebra* (5th). Wellesley-Cambridge Press.
- Yi, L., Li, C., & Guo, J. (2020). CPI for Runtime Performance Measurement: The Good, the Bad, and the Ugly. *IEEE International Symposium on Workload Characterization (IISWC 2020)*, 15-25. <https://doi.org/10.1109/IISWC50251.2020.00011>